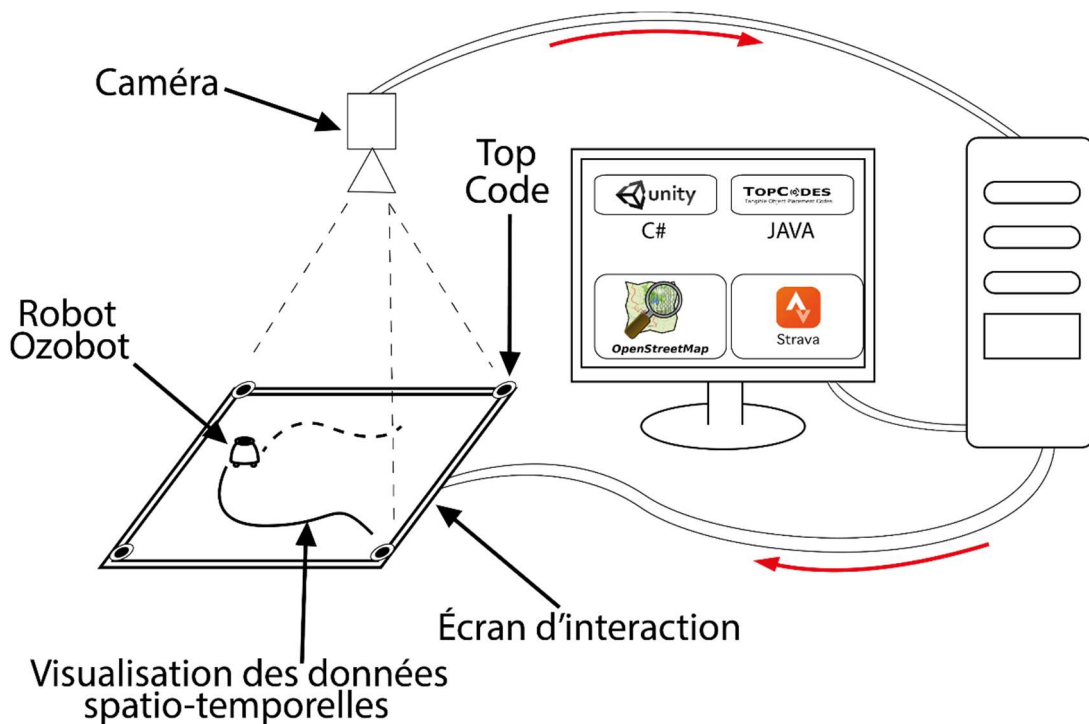




Exploration de données spatio-temporelles avec des objets tangibles autonomes

Réalisé par Sabine Cassat, étudiante à
l'école d'ingénieurs UPSSITECH

Encadré par Michael Lauer, maîtres de
conférence au LAAS-CNRS

Encadré par Emmanuel Dubois et Marcos
Serrano, maîtres de conférence à l'IRIT

Remerciements

Je voudrais exprimer ma gratitude à tous ceux et celles qui m'ont donné l'opportunité de réaliser ce stage à l'Institut de Recherche en informatique de Toulouse (IRIT).

Je remercie avant tous mes encadrants, Emmanuel Dubois et Marcos Serrano, maîtres de conférences à l'IRIT, pour m'avoir accueilli au sein de leur équipe ELIPSE pour réaliser ce projet de fin d'études. Je remercie également l'ensemble de l'équipe ELIPSE et tous les membres du projet néOCampus pour leur accueil chaleureux. Je remercie tout particulièrement l'ensemble des stagiaires qui ont accepté de tester mon application et de faire des retours réguliers sur celle-ci afin de l'améliorer.

J'aimerais particulièrement remercier également mon tuteur académique Michael Lauer, maître de conférences au LAAS-CNRS, ainsi que Philippe Truillet, mon responsable d'année à l'école d'ingénieurs UPSSITECH et maître de conférence à l'IRIT, pour leur soutien et leurs conseils.

Résumé

Dans le cadre de mon stage de dernière année, j'eus l'opportunité de réaliser un projet de 21 semaines à l'IRIT proposant une solution innovante pour l'exploration de données spatio-temporelles (DST) à l'aide d'objets tangibles autonomes. Les objectifs de ce stage furent de combiner des interfaces tangibles robotisées et des visualisations 2D sur une table interactive pour permettre l'accessibilité, la maniabilité et l'interprétation de DST. Au cours de ce stage, une publication fut soumise et acceptée à la conférence « MultimodalVis 2018 » (voir document annexe « Publication à la conférence « MultimodalVis 2018 » nommée « A Novel Interaction Paradigm For Exploring Spatio-Temporal Data » »), et une autre publication fut soumise à la conférence « IHM 2018 » dont les retours nous parviendront au début du mois de septembre (voir document annexe « Publication à la conférence « IHM 2018 » nommée « Une nouvelle approche pour l'exploration de données spatio-temporelles » »).

La réalisation de ce stage s'inscrit dans le projet néOCampus qui regroupe aujourd'hui 11 laboratoires partenaires combinant leurs compétences dans différents domaines (informatique, matériaux, électricité, écologie, etc.) pour construire le campus universitaire de demain : intelligent, innovant et durable. De ce fait, j'eus également l'opportunité d'intervenir lors des différentes réunions, présentations et salons néOCampus tout au long de mon stage pour présenter non seulement notre projet, mais également d'autres projets néOCampus à la communauté scientifique et industrielle.

Ce document a pour but de décrire l'ensemble du travail qui a été réalisé au cours de cette expérience professionnelle constituant mon stage de fin d'étude à l'école d'ingénieurs UPSSITECH. Dans un premier temps, nous ferons une présentation de l'IRIT et de l'équipe ELIPSE. Puis, une présentation des objectifs et des différentes étapes du stage sera effectuée. Par la suite, nous introduirons les concepts et les différentes technologies utilisées pour la réalisation de ce projet. Cela sera suivi d'une description des différents modes d'utilisation de l'application développée au cours de ce stage selon deux scénarios clés différents : l'un incluant le suivi d'un cycliste au cours de son entraînement, et l'autre incluant le suivi d'un étudiant parcourant le campus de l'Université Paul Sabatier III en passant à proximité des différents laboratoires partenaires de néOCampus. Ensuite, une brève explication de la mise en œuvre de fichiers exécutables de l'application et de la documentation du projet sera énoncée. Enfin, nous concluons sur les différents accomplissements et sur les retours humains de ce projet.

Table des matières

Remerciements	3
Résumé	5
Présentation de l'IRIT	11
I. L'IRIT	11
II. Équipe ELIPSE	11
Introduction.....	13
I. Objectifs de stage	13
II. Différentes étapes du stage	13
Description technique des concepts, des outils logiciels et matériels.....	15
I. Extraction des données spatio-temporelles.....	17
1. Strava.....	17
2. Tahoe - OpenStreetMap.....	17
II. Modélisation du cube spatio-temporel à partir des données spatio-temporelles	18
1. Choix de l'environnement de développement logiciel	19
2. Modélisation du cube spatio-temporel dans Unity.....	20
III. Choix du type de robot utilisé pour notre système.....	20
IV. Localisation du robot sur l'écran d'interaction	22
1. Localisation du robot sur la surface de l'écran.....	22
2. Détection de l'orientation du robot	24
3. Détection de l'élévation du robot au-dessus de l'écran	24
4. Création d'un serveur de communication avec Unity.....	25
V. Génération de trajectoire du robot.....	25
VI. Création de l'interface graphique à partir du cube spatio-temporel.....	27
1. Boutons de paramétrage et de sélection du mode d'exploration	28
2. Légendes de couleur autour du robot.....	30
3. Modification de la vitesse du robot	31
Modes d'exploration des DST	33
I. « Manual Data Mining »	33
II. « Automatic Data Mining »	35
III. « Elevation-Related Interest Segment Selection »	36
IV. « User-Defined Interest Segment Selection »	37
V. Adaptation du programme d'exploration pour le scénario néOCampus.....	38
Code source, exécutable et documentation	41

I. Installations et configurations	41
II. Lancement de l'application développée	43
III. Documentation du code source	43
IV. Vidéos de démonstration	44
Conclusion	45
Références	47

Table des figures

Figure 1 : Schéma du concept de CST.....	15
Figure 2 : Schéma du système de visualisation et d'interaction pour l'exploration de DST	16
Figure 3 : Capture d'écran du parcours sportif réalisé par un cycliste enregistré sous Strava.....	17
Figure 4 : Capture d'écran montrant le fonctionnement du programme Tahoe pour l'extraction des cartes géographiques selon des coordonnées GPS.....	18
Figure 5 : Captures d'écran sous plusieurs angles de vue de la modélisation du CST avec Processing	19
Figure 6 : Captures d'écran sous plusieurs angles de vue de la modélisation du CST avec Unity	20
Figure 7 : Vue éclatée du robot Ozobot Bit.....	21
Figure 8 : Capture d'écran de la documentation du robot Ozobot [9]	22
Figure 9 : Schémas représentatifs de l'erreur angulaire en X et Y à la formation du repère orthogonal de référence	23
Figure 10 : Schéma représentatif de l'erreur angulaire observé en différents points de l'écran d'interaction après la création du nouveau repère orthogonal.....	23
Figure 11 : Schéma représentatif de l'erreur angulaire observé en différents points de l'écran d'interaction après mise en place des algorithmes de correction.....	24
Figure 12 : Schéma descriptif de l'algorithme de génération de trajectoire circulaire	26
Figure 13 : Placement du robot tangent à son cercle de rotation	27
Figure 14 : Optimisation de trajectoire selon la distance à parcourir.....	27
Figure 15 : Capture de l'interface graphique du système	28
Figure 16 : Paramétrage du programme d'exploration des DST via l'interface graphique.....	29
Figure 17 : Sélection du mode d'exploration des données via l'interface graphique du programme d'exploration des DST.....	30
Figure 18 : Photo du parcours des DST par le robot Ozobot avec l'inclusion d'informations complémentaires.....	30
Figure 19 : Photos d'une démonstration du mode d'exploration « Manual Data Mining »	34
Figure 20 : Photos d'une démonstration du mode d'exploration « Elevation-Related Interest Segment Selection »	36
Figure 21 : Photos d'une démonstration du mode d'exploration « User-Defined Interest Segment Selection »	37
Figure 22 : Demande d'autorisation de Windows pour permettre à l'application de passer le pare-feu présent sur l'ordinateur hôte	42
Figure 23 : Capture d'écran de la fenêtre de lancement du programme d'exploration des DST	42
Figure 24 : Capture d'écran de la documentation d'une des classes du projet (générée avec Doxygen)	44

Table des tableaux

Tableau 1 : Récapitulatif des informations affichées lors de la saisie du robot au cours du mode d'exploration « Manual Data Mining »	35
Tableau 2 : Récapitulatif des informations affichées lors de la saisie du robot au cours du mode d'exploration « Elevation-Related Interest Segment Selection »	37
Tableau 3 : Récapitulatif des informations affichées lors de la saisie du robot au cours du mode d'exploration « User-Defined Interest Segment Selection » (scénario cycliste).....	38
Tableau 4 : Récapitulatif des informations affichées lors de la saisie du robot au cours du mode d'exploration « User-Defined Interest Segment Selection » (scénario néOCampus)	39

Table des documents annexes

Document 1 : État de l'art sur l'exploration de données spatio-temporelles

Document 2 : Répartition des tâches dans le temps à travers un diagramme de Gantt

Document 3 : Spécifications des robots de petites tailles étudiés pour notre système d'exploration de données spatio-temporelles

Document 4 : Publication à la conférence « MultimodalVis 2018 » nommée « A Novel Interaction Paradigm For Exploring Spatio-Temporal Data »

Document 5 : Publication à la conférence « MultimodalVis 2018 » nommée « A Novel Interaction Paradigm For Exploring Spatio-Temporal Data » (poster)

Document 6 : Publication à la conférence « IHM 2018 » nommée « Une nouvelle approche pour l'exploration de données spatio-temporelles »

Présentation de l'IRIT

I. L'IRIT

L'Institut de Recherche en Informatique de Toulouse (IRIT) est l'une des plus importantes Unités Mixtes de Recherche (UMR) au niveau national. Il est composé aujourd'hui de plus de 700 membres, permanents et non permanents. Il est structuré en 7 départements de recherche qui regroupent 24 équipes :

- Département Signaux et Images (SI),
- Département Science des Données (SD),
- Département Interaction et Intelligence Collective (ICI),
- Département Intelligence Artificielle (IA),
- Département Calcul Intensif, Simulation et Optimisation (CISO),
- Département Architecture Systèmes et Réseaux (ASR),
- Département Fiabilité des Systèmes et des Logiciels (FSL).

De par son caractère multitutelle (CNRS, Toulouse INP, Universités toulousaines), son impact scientifique et ses interactions avec les autres domaines, l'IRIT constitue une des forces structurantes du paysage de l'informatique et de ses applications dans le monde du numérique.

II. Équipe ELIPSE

L'équipe ELIPSE a pour objet d'étude l'Interaction Homme-Machine avec une approche interdisciplinaire importante et indispensable à ce domaine. En effet, en associant les disciplines telles que l'interaction homme-machine, l'informatique, la médecine, les neurosciences, l'ergonomie et la psychologie, cette équipe a les moyens de faire converger les fondements théoriques liés à l'humain et ses interactions en travaillant sur la modélisation des interactions, les méthodes et processus de conception, les théories de la perception, de l'action et de la cognition. L'équipe est également en mesure d'aller jusqu'au déploiement et l'évaluation de situations d'interactions complexes et variées comme par exemple pour les technologies de suppléance pour le handicap, les systèmes interactifs mobiles, les espaces d'interaction spatialisée tels que la réalité augmentée, les interactions tangibles, l'informatique ambiante, etc.

Introduction

Des ensembles de DST sont générés quotidiennement, que ce soit pour étudier l'expansion d'une maladie, les phases migratoires de la faune ou la consommation énergétique d'un campus. Les visualisations de tels jeux de données incluent généralement des rendus 2D, des visualisations animées ou des représentations 3D comme des cubes spatio-temporels (voir document annexe « État de l'art sur l'exploration de données spatio-temporelles »). Face à des environnements de données aussi riches et complexes, on ne sait pas comment les systèmes actuels supportent l'observation et la construction de sens lorsque les individus doivent interagir avec de tels ensembles de données, les analyser et les explorer. Cependant, étant donné la complexité et le volume de ces jeux de données, de telles explorations sont cruciales pour comprendre les modèles mathématiques sous-jacents. Les solutions explorées jusqu'ici incluent la souris, les interfaces tangibles et/ou tactiles qui présentent pour chacune d'elles des limitations (voir document annexe « État de l'art sur l'exploration de données spatio-temporelles »).

I. Objectifs de stage

L'objectif principal de ce stage est donc de développer une approche radicalement différente de l'exploration de DST en associant l'interaction tangible, la robotique et l'interaction tactile. Notre approche consiste à combiner l'utilité inhérente des objets tangibles autonomes, tels que des robots, pour représenter des entités dans un ensemble de DST, avec l'opportunité d'enrichir les informations qu'ils représentent dans l'espace et le temps en utilisant des visualisations sur une table interactive. Le but sera de faire progresser la compréhension des ensembles de DST pour les représenter avec des technologies émergentes, de faire avancer les normes d'analyse de données actuelles et de répondre à un besoin clé en analyse visuelle.

II. Différentes étapes du stage

La réalisation de ce stage fut découpée en plusieurs étapes (voir document annexe « Répartition des tâches dans le temps à travers un diagramme de Gantt »). Dans un premier temps, une étude bibliographique fut effectuée afin d'étudier les différentes formes de visualisation et d'interaction existantes avec les données spatio-temporelles, ainsi que les différents robots de petite taille existants et utilisables dans le cadre de notre projet. Par la suite, une phase de conception fut entamée. Un ensemble de scénarios et de sketchings vidéo furent réalisés afin d'évaluer avant la phase de développement logiciel les moyens d'interaction et d'exploration des DST imaginées dans cette phase de conception. De cette manière, il fut possible de définir une méthode de visualisation et de représentation des données spatio-temporelles, ainsi qu'un ensemble d'opérations possibles à travers ce système de visualisation afin d'explorer plus en détail les ensembles de données étudiés. Enfin, une phase de développement logiciel fut initiée afin de créer un prototype de l'application en question. Tout au long de cette phase, mes encadrants et moi-même avons pris en compte des retours utilisateurs (notamment de la part des autres stagiaires) afin d'en améliorer son utilisabilité, tout en ajoutant des fonctionnalités au fur et à mesure que le système de visualisation proposait de plus en plus de modes d'exploration des DST. Pour conclure ce stage, un ensemble de vidéo de démonstration

fut réalisé et une publication décrivant notre prototype fut soumise à la conférence « IHM 2018 » (voir les documents annexes « Publication à la conférence « MultimodalVis 2018 » nommée « A Novel Interaction Paradigm For Exploring Spatio-Temporal Data » » et « Publication à la conférence « IHM 2018 » nommée « Une nouvelle approche pour l'exploration de données spatio-temporelles » »).

Description technique des concepts, des outils logiciels et matériels

Parmi l'ensemble des techniques de visualisation de représentation des DST (voir le document annexe « État de l'art sur l'exploration de données spatio-temporelles »), nous avons choisi d'utiliser le concept de cube spatio-temporel (CST). En effet, cette représentation de données est relativement intuitive. Deux dimensions contiennent les informations spatiales des DST, tandis qu'une dimension contient les informations temporelles, ces deux types d'informations étant indissociables l'un de l'autre.

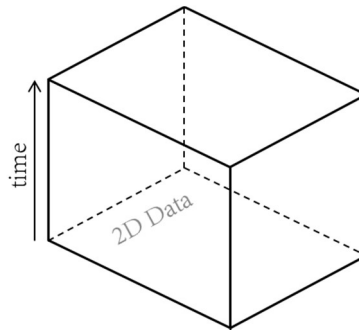


Figure 1 : Schéma du concept de CST

De nombreux articles étudient les différentes opérations possibles sur cet espace de représentation de donnée [4, 6, 10, 11]. On retrouve cinq classifications d'opérations [1] :

- « L'extraction » qui consiste à sélectionner un sous-ensemble d'un objet espace-temps,
- « L'aplatissement » qui consiste à agréger un objet espace-temps en un objet espace-temps de dimension inférieure,
- Le « remplissage » qui consiste à transformer un ensemble d'objets spatio-temporels déconnectés en un objet spatio-temporel entièrement connecté,
- La « transformation géométrique » qui consiste à transformer spatialement un objet espace-temps sans en changer son contenu, et enfin,
- La « transformation de contenu » qui consiste à modifier le contenu d'un objet espace-temps sans affecter sa géométrie.

Le but ultime des opérations de cubes spatio-temporelles est de transformer un cube spatio-temporel en un objet spatio-temporel dont la représentation est compatible avec la forme du support employé pour véhiculer l'information. Dans notre cas, nous avons choisi d'utiliser un écran 2D posé à plat comme support de visualisation et d'interaction pour parcourir l'ensemble des DST étudiées. En effet, cet écran sera utilisé pour visualiser des coupes du cube spatio-temporel selon un instant donné ou un intervalle de temps donné. C'est sur ce dispositif de support de visualisation que nous utilisons des interfaces tangibles autonomes, tels que des robots, pour représenter l'entité clé qu'est l'objet des DST.

Afin de pouvoir localiser un robot sur l'écran d'interaction, nous avons choisi d'utiliser une librairie développée en JAVA appelée TopCode qui permet de localiser sur une image des symboles circulaires particuliers appelés TopCode. De ce fait, en plaçant une caméra au-dessus de l'écran, des TopCodes aux quatre coins de l'écran et un autre TopCode sur le dessus du robot, il nous est donc possible d'estimer sa position directement sur l'écran d'interaction tout en permettant une visualisation des DST sur la surface de l'écran. C'est sur cette même surface que le robot pourra se déplacer et interagir en toute autonomie.

Dans le but de tester notre prototype, nous avons utilisé des DST simples provenant d'un parcours sportif effectué par un cycliste aux alentours de Toulouse. Cet itinéraire fut extrait d'un fichier GPX fourni par la plateforme web Strava qui fut utilisée pour générer les DST via le smartphone du cycliste. De plus, nous avons utilisé l'API OpenStreetMap pour extraire les cartes géographiques associées au parcours de ce cycliste.

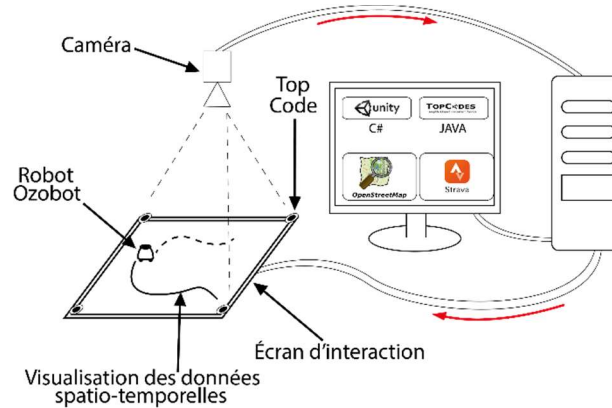


Figure 2 : Schéma du système de visualisation et d'interaction pour l'exploration de DST

En résumé, à partir des DST extraites de **Strava** et des cartes géographiques d'**OpenStreetMap**, il fut possible de modéliser le CST en 3D sous **Unity** dont les coupes temporelles furent affichées sur l'**écran d'interaction** 2D posé à plat. C'est sur cet écran que le ou les **robots** peuvent interagir avec les données affichées, ou représenter une interface tangible d'interaction pour aider l'utilisateur à interagir avec le système. En plaçant une **caméra** au-dessus de l'écran, un programme **JAVA** est alors en charge de localiser le ou les robots présents sur l'écran d'interaction à l'aide la librairie **TopCode**, information qu'il communiquera à Unity, à l'aide d'un serveur local, pour mettre à jour les informations disponibles sur l'écran d'interaction (que ce soit les informations demandées par l'utilisateur, ou le chemin que chacun des robots présents sur l'écran doit suivre pour réaliser la tâche demandée par l'utilisateur).

Une description plus détaillée de chacun de ces outils est explicitée dans ce document afin de comprendre comment le système fut construit et comment l'utilisateur peut interagir avec celui-ci pour explorer les DST qu'ils souhaitent étudier.

I. Extraction des données spatio-temporelles

1. Strava

Comme mentionné précédemment, nous avons utilisé la plateforme web Strava pour générer les DST. Strava est à la fois un site internet et une application mobile qui sont utilisés pour enregistrer des performances sportives via GPS. Strava est capable d'enregistrer les traces de plus de 30 sports différents.

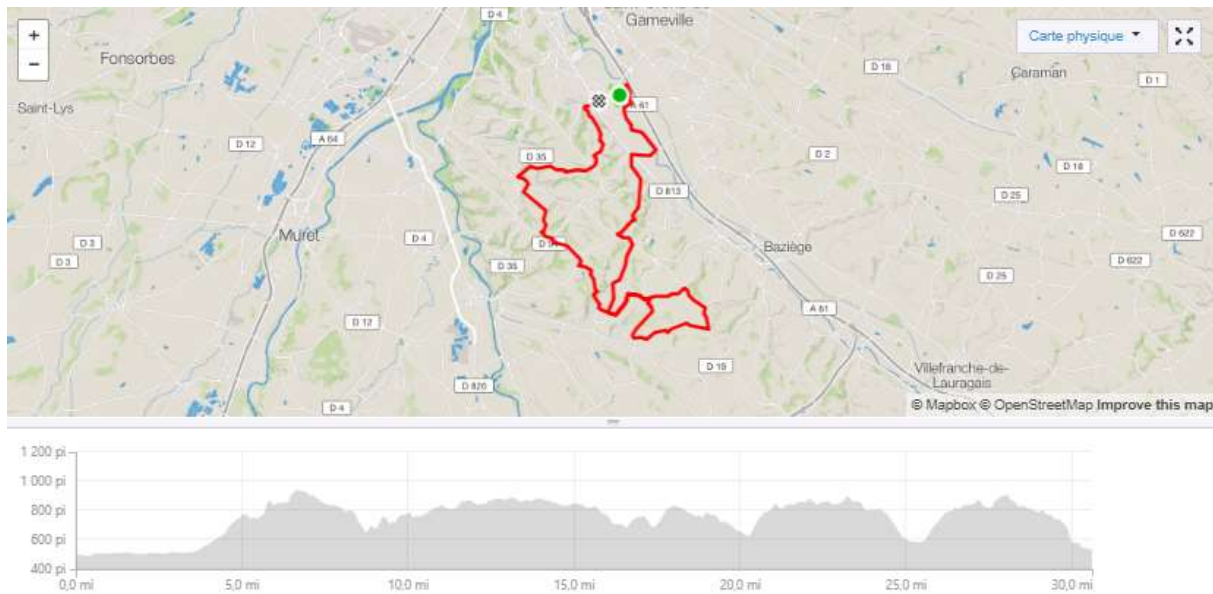


Figure 3 : Capture d'écran du parcours sportif réalisé par un cycliste enregistré sous Strava

Dans notre cas, nous avons enregistré à l'aide d'un smartphone le parcours d'un cycliste réalisant son parcours sportif hebdomadaire (voir Figure 3). Ce n'est qu'une fois ces données enregistrées que nous avons pu extraire un fichier GPX contenant les informations spatiales et temporelles de son entraînement. Parmi ces informations, on retrouve sa position en latitude, longitude et élévation à chaque instant où les mesures furent effectuées (environ toutes les 4 à 8 secondes pendant 2h13).

2. Taho - OpenStreetMap

À partir des DST enregistrées et extraites depuis Strava, il est alors possible de récupérer les cartes géographiques associées à ces données. Pour cela, nous avons employé le programme open source Taho utilisant l'API OpenStreetMap pour capturer les cartes selon un cadre de coordonnées GPS (voir Figure 4).

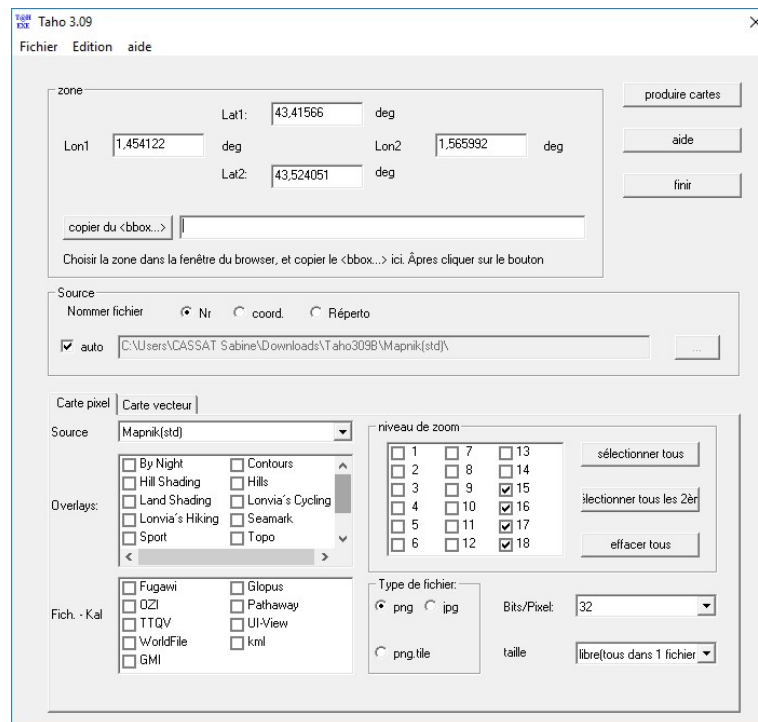


Figure 4 : Capture d'écran montrant le fonctionnement du programme Tahoe pour l'extraction des cartes géographiques selon des coordonnées GPS

En donnant au programme une latitude minimum et maximum, ainsi qu'une longitude minimum et maximum, une carte sous format PNG sera alors générée selon l'espace géographique définie par l'utilisateur. L'utilisateur devra spécifier s'il souhaite obtenir cette carte sur plusieurs fichiers de mêmes tailles ou un seul fichier. Il devra également fournir le niveau de zoom qu'il souhaite appliquer à l'obtention de cette carte modifiant donc la résolution de l'image obtenue.

Attention, dans notre cas, il est fortement recommandé de définir un cadre de coordonnées GPS dont le résultat donnera une carte carrée. Comme la latitude est définie entre -90° et 90° , et la longitude est définie entre -180° et 180° , il faudra respecter l'équation suivante : $(longitude_{max} - longitude_{min}) = (latitude_{max} - latitude_{min}) \times 2$. Ce seront les coordonnées utilisées pour créer le cadre de coordonnées GPS et qu'il faudra communiquer à l'application développée sous Unity pour que la création du CST puisse s'effectuer sans déformation. Dans le cas où l'utilisateur ne respecte pas cette équation, le programme fonctionnera tout de même, mais la carte géographique semblera légèrement déformée (allongée verticalement ou horizontalement) lors de son importation dans Unity.

II. Modélisation du cube spatio-temporel à partir des données spatio-temporelles

Une fois les DST extraites et les cartes géographiques téléchargées, il est alors possible de générer le CST. Pour cela, nous avons étudié différentes possibilités de modéliser logiquement cette structure de données.

1. Choix de l'environnement de développement logiciel

a. Processing

Dans un premier temps, nous nous sommes intéressés à la modélisation de ce cube sous Processing. Processing est une bibliothèque JAVA et un environnement de développement open source. Il est tout particulièrement utilisé dans le cadre de création graphique interactive. L'atout majeur de Processing est sa simplicité d'utilisation, notamment dans la syntaxe du langage de programmation pour le développement de programmes. Adapté à la création graphique, Processing réclame moins d'efforts que JAVA pour effectuer des tâches simples telles que la modification d'une animation à intervalle régulier (qui permet des créations animées). Ses fonctionnalités sont limitées aux besoins des créateurs d'images 2D et 3D générées par programmation, mais peuvent être étendues, par le biais de modules externes, à la capture d'un flux vidéo, à la génération et à la manipulation de son, à l'interfaçage des ports d'entrées-sorties, etc.

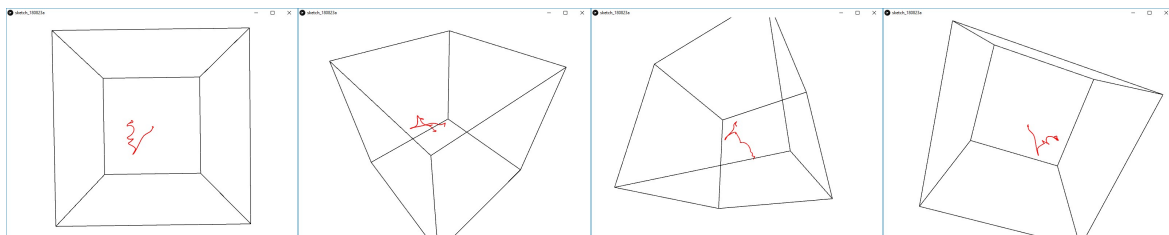


Figure 5 : Captures d'écran sous plusieurs angles de vue de la modélisation du CST avec Processing

Bien qu'en utilisant des bibliothèques créées pour générer des formes 3D, au vu de la complexité des DST à traiter, il fut difficile de créer l'ensemble des objets 3D de manière efficace. De plus, le rendu obtenu à partir des caméras virtuelles ne fut pas celui escompté initialement (voir Figure 5) et demanda beaucoup de ressources au PC pour compiler et lancer l'animation sous Windows. D'autre part, pour la poursuite de ce projet, il fut imaginé d'utiliser la réalité augmentée comme une autre source de visualisation. Un environnement de développement tel que Processing ne permet pas d'inclure le développement d'une application en réalité augmentée pour le casque disponible au laboratoire, le casque Hololens. Par conséquent, nous avons étudié une autre possibilité : Unity3D.

b. Unity

Unity est un moteur de jeu très répandu dans le monde des jeux vidéo du fait de sa rapidité pour le prototypage. Un atout d'Unity, important dans notre cas, est qu'il est capable d'exporter sous forme d'exécutable les applications sur de nombreuses plateformes (smartphone, PC, etc.), y compris un casque de réalité augmentée. Il possède une interface d'intégration d'objets et de scripts très intuitive pour le développement d'objets virtuels animés. Contrairement à Processing, il permet de séparer l'interface graphique (ou le canevas) de l'animation des objets virtuels qui autorisent l'utilisateur à structurer son application selon le « design pattern » Modèle-Contrôleur-Vue. Il permet aussi d'importer des objets virtuels depuis d'autres sources de modélisation 3D (Blender, etc.) ou des textures d'objets à partir d'images. D'autre part, une licence gratuite appelée « Personal » permet d'accéder à toutes les fonctionnalités du moteur de jeu. Unity possède également une grande communauté prête à apporter son aide pour toute personne débutante sur cet environnement de développement. En raison de tous ces arguments, nous avons choisi d'utiliser cet environnement de développement logiciel pour la création de notre prototype de visualisation et d'interaction avec des

DST, à défaut d'autres outils de développement logiciel pour des environnements virtuels 3D comme Java3D, TinkerCAD ou 3D Slash.

2. Modélisation du cube spatio-temporel dans Unity

Le cube est une forme géométrique de base proposée par Unity. Il est possible de le modifier de manière à ne repérer uniquement que ses bords et ainsi pouvoir étudier les DST contenues dans celui-ci. Seulement, à la création de ce cube, les DST ne sont pas encore disponibles puisqu'elles sont contenues dans le fichier GPX extrait depuis la plateforme Strava. De même pour la carte géographique d'OpenStreetMap qui a besoin d'être importée dans le programme d'exploration des DST. Par conséquent, un script fut créé afin d'extraire les données du fichier GPX contenant les informations spatiales et temporelles du cycliste, de créer un objet de type courbe représentant l'itinéraire en 3D de son parcours sportif. Enfin, un autre script fut en charge d'ajouter la carte géographique à l'origine du repère spatial du cube spatiotemporel afin de pouvoir visualiser l'emplacement sur une carte de chacun des points spatio-temporels en prenant compte des éventuelles marges spatiales décrites dans la sous-section « Tahoe – OpenStreetMap » de la section « Extraction des données spatio-temporelles ».

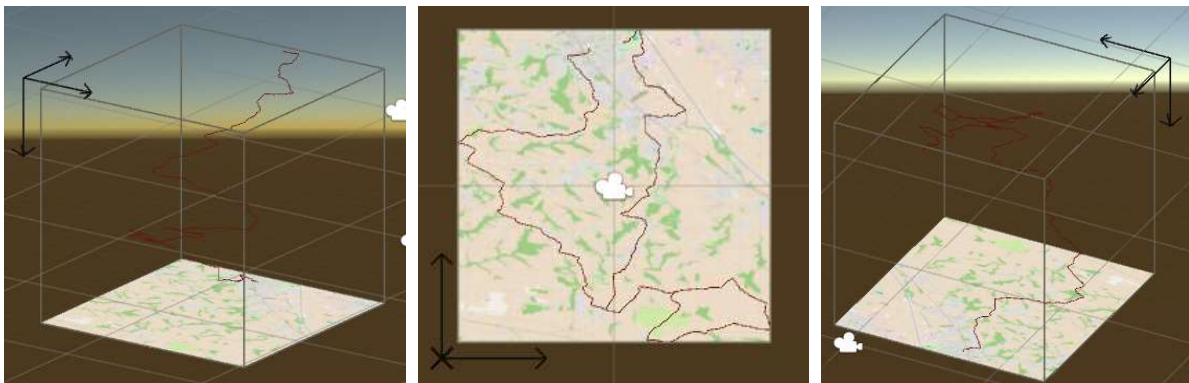


Figure 6 : Captures d'écran sous plusieurs angles de vue de la modélisation du CST avec Unity

III. Choix du type de robot utilisé pour notre système

Ce projet étant initialement voué à évoluer par la suite en sujet de thèse, une étude des différents robots de petite taille présents sur le marché ou utilisés en recherche fut effectuée. Parmi les robots en question, on retrouve : les robots Zooids [5], l'EcoBe! [3], le robot Cellulo [7, 8] et les robots Ozobot [9] (Bit et Evo). Chacun d'eux présente des caractéristiques particulières (voir le document annexe « Spécifications des robots de petites tailles étudiés pour notre système d'exploration de données spatio-temporelles »).

Dans notre cas, nous avons besoin d'un robot de petite taille, avec au minimum une heure d'autonomie, que l'on puisse contrôler depuis notre système de visualisation et qui soit accessible à un coût raisonnable. Des robots Ozobot Bit étant déjà disponibles dans le laboratoire ayant été utilisés

sur d'autres projets, il fut décidé d'utiliser ces robots dans un premier temps pour tester notre prototype.

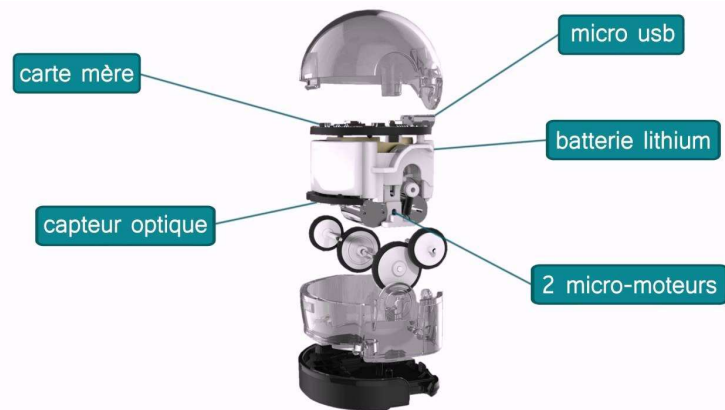


Figure 7 : Vue éclatée du robot Ozobot Bit

Le robot Ozobot Bit est un robot d'allure sphérique avec deux roues munies de micromoteurs, une carte mère, un micro port USB, une batterie lithium et des capteurs optiques permettant de détecter des codes couleurs et des bordures de lignes (voir Figure 7). Celui-ci est programmable via la plateforme web « Ozoblocky », dans un langage très similaire au langage scratch.

Ce robot présente tout de même des limitations qui eurent un impact sur le développement de notre prototype. En effet, celui-ci fut conçu pour l'apprentissage de la programmation pour les enfants. Par conséquent, peu de fonctions sont disponibles pour programmer le robot. De plus, si l'utilisateur souhaite programmer le robot, il doit bien faire attention à le programmer en prenant compte des événements et des fonctions bloquantes qui ne sont pas nécessairement détaillées dans la documentation de « Ozoblocky ».

De plus, le robot Ozobot peut suivre des lignes de différentes couleurs. Donc il n'est possible de l'arrêter qu'avec une détection de la couleur blanche par les capteurs optiques de celui-ci. Par conséquent, à chaque fois que le robot fut placé sur l'écran d'interaction, il fut nécessaire de faire apparaître un cercle blanc plein autour du robot de manière à ce qu'il n'interprète pas les couleurs autour de lui comme étant des lignes à détecter et à suivre.

D'autre part, d'après la documentation (voir Figure 8), ce robot n'est pas capable de suivre des lignes dont la trajectoire inclut des angles aigus. Or, les DST n'étant pas créées en prenant compte de cette limitation, il peut arriver que le robot s'arrête au niveau des angles trop aigus qu'il détecte alors comme une fin de ligne.

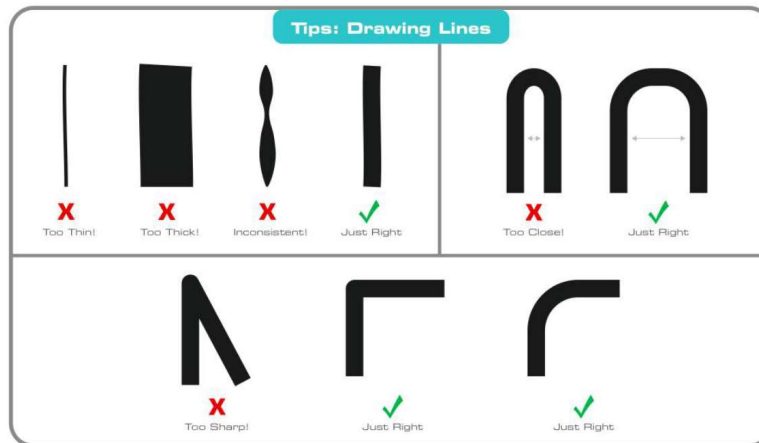


Figure 8 : Capture d'écran de la documentation du robot Ozobot [9]

Enfin, d'après cette même documentation, les capteurs optiques du robot sont très sensibles aux variations de luminosité, ce qui peut être corrigé en recalibrant les capteurs du robot à chaque nouvelle utilisation du robot ou à chaque changement de luminosité autour du robot.

IV. Localisation du robot sur l'écran d'interaction

La localisation du robot sur l'écran est primordiale pour lui permettre d'interagir avec les données et l'utilisateur. Un programme fut donc développé pour estimer la position du robot selon quatre symboles circulaires visuels, nommés des TopCodes, placés sur les quatre bords de l'écran. Pour cela, la librairie TopCode fut utilisée. Cependant, celle-ci s'est montrée être peu robuste aux déformations géométriques de la caméra, au placement de celle-ci au-dessus de l'écran d'interaction et aux changements d'altitude des différents objets à repérer. Nous avons donc mis en place quelques ajustements permettant de corriger les erreurs en position et en orientation des différents objets à repérer à l'intérieur des quatre TopCode placés sur l'écran.

1. Localisation du robot sur la surface de l'écran

Dans un premier temps, nous avons tenté de créer un repère orthogonal à partir de trois des TopCode placés sur les quatre bords de l'écran. Seulement, au vu des déformations géométriques de la caméra, des erreurs angulaires furent observées.

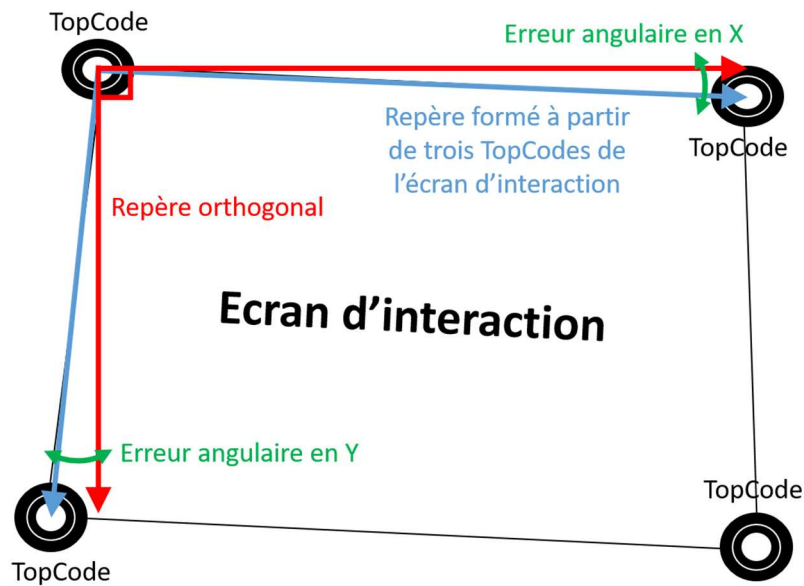


Figure 9 : Schémas représentatifs de l'erreur angulaire en X et Y à la formation du repère orthogonal de référence

En effet, comme on peut l'observer sur la Figure 9, il n'est pas possible de faire un angle droit parfait entre les trois TopCode (en bas à gauche, en haut à gauche et en haut à droite de l'écran). Même en prenant trois autres TopCode pour créer ce nouveau repère, des erreurs angulaires furent observées, cela faussant l'estimation de position du robot à l'intérieur de ce repère (voir Figure 10).

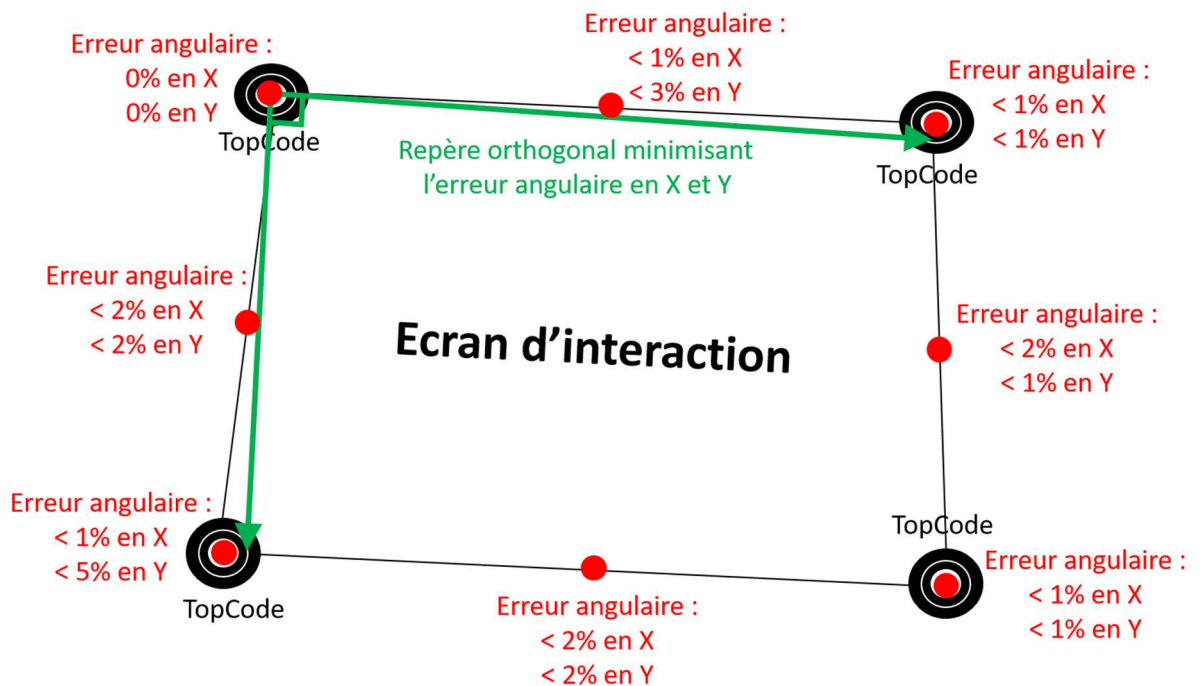


Figure 10 : Schéma représentatif de l'erreur angulaire observé en différents points de l'écran d'interaction après la création du nouveau repère orthogonal

On remarque sur la Figure 10 que l'erreur en position est nulle lorsque le robot est proche de l'origine du repère. Par conséquent, nous avons choisi de placer un repère orthogonal sur chacun des TopCodes présents sur les coins de l'écran tout en minimisant l'erreur angulaire sur chacun des

vecteurs directeurs de ces repères. Ainsi lorsque le robot se déplace entre l'origine de ces quatre repères, sa position et son orientation sont exprimées selon quatre systèmes de coordonnées. Il suffit alors d'effectuer une rotation et une translation de chaque repère pour pouvoir les estimer selon une origine commune.

Nous avons observé précédemment que plus le robot est proche de l'origine d'un repère, plus l'estimation de sa position est précise. Nous avons donc réalisé une moyenne pondérée de la position du robot en fonction de la distance entre le robot et chacune des origines des repères. Comme on peut le voir Figure 11, cela permet grandement de minimiser les erreurs que nous observions Figure 10.

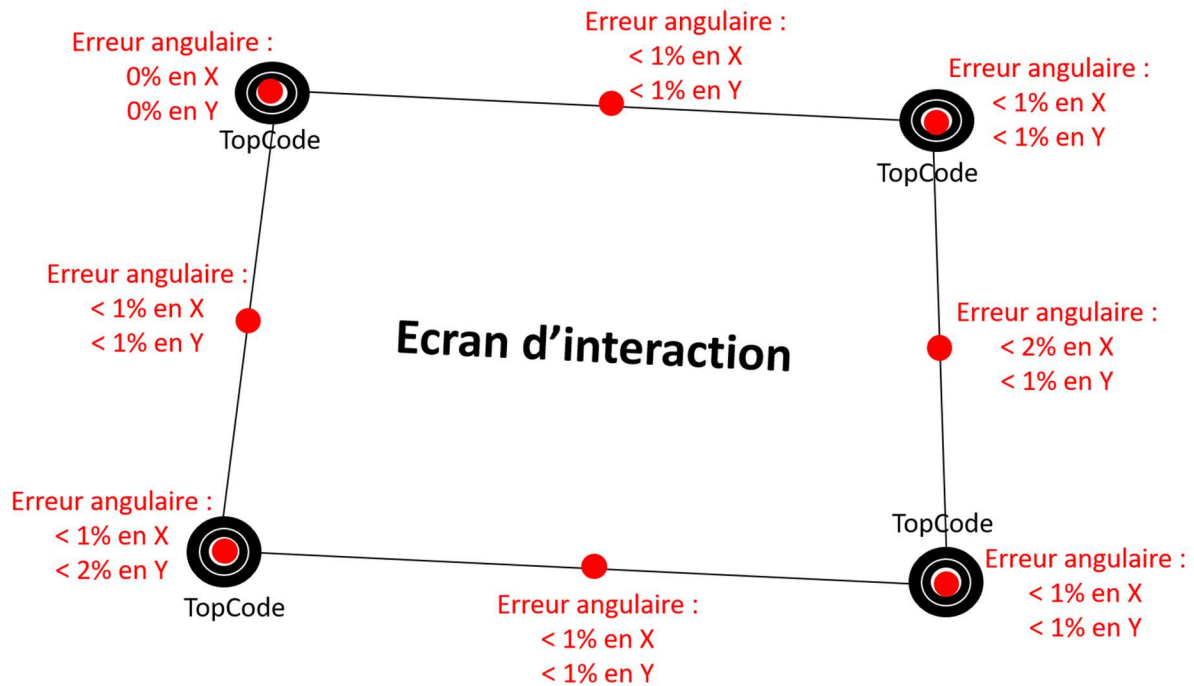


Figure 11 : Schéma représentatif de l'erreur angulaire observée en différents points de l'écran d'interaction après mise en place des algorithmes de correction

2. Détection de l'orientation du robot

Grâce à la librairie TopCode, il fut possible de déterminer l'orientation de chacun des TopCode présents sur les images de la caméra. Toutefois, une erreur importante fut remarquée. En effet, les valeurs retournées par la fonction permettant d'extraire l'orientation d'un TopCode sont comprises entre -7° et 350° . Bien que cela ne soit pas une solution à long terme, un simple produit en croix fut effectué pour ramener les valeurs angulaires entre 0 et 360 (générant une légère erreur de 0.008° à chaque mesure) : $\theta = \frac{(-\theta_{reçu} + 7) \times 360}{357}$

3. Détection de l'élévation du robot au-dessus de l'écran

À tout moment, le robot peut être saisi par l'utilisateur de manière à afficher davantage d'informations lors de l'exploration des données. Nous avons donc dans un premier temps mis en place

des petites marches en carton pour chacun des TopCode du bord de l'écran de la taille de la hauteur du robot utilisé, soit environ 3 centimètres. De cette manière, lorsque le robot est posé sur l'écran d'interaction, l'ensemble des TopCode (celui du robot et ceux du bord de l'écran) sont placés à une même hauteur. L'ensemble des TopCode ayant tous la même largeur, cela nous permet d'estimer en fonction de la largeur détectée du TopCode du robot à quelle hauteur par rapport au TopCode du bord de l'écran il est soulevé par l'utilisateur. Il est alors possible de définir un certain nombre de paliers d'élévation qui pourront être interprétés par le système de visualisation des DST comme une action utilisateur.

4. Création d'un serveur de communication avec Unity

Au lancement du programme JAVA de localisation, un serveur local est exécuté sur l'ordinateur hôte. Pour cela, le programme de localisation utilise un socket de communication JAVA « ServerSocket » qui ouvre une connexion sur le port 8080 de l'ordinateur hôte. À la construction de ce socket, une fonction bloquante est appelée permettant d'accepter la connexion avec un socket client qui en fera la demande. Ce socket de communication client sera créé dans le programme d'exploration des données implémenté sous Unity et devra faire la demande de connexion au serveur de localisation pour recevoir la position et orientation du robot sur l'écran d'interaction.

À chaque fois que le TopCode du robot est repéré sur l'image, sa position et son orientation sont estimées selon les algorithmes précédemment expliqués et sont ensuite envoyées au programme Unity. Lorsque le robot n'est pas détecté sur l'image, aucun message n'est envoyé. Le programme Unity possède un socket client dont la fonction de réception est munie d'un compteur « timeout ». Ainsi, il vérifie pendant 100 millisecondes au maximum si la position du robot est détectée, puis continue la mise à jour des différents objets et scripts du programme. Si au bout de 500 millisecondes Unity ne reçoit toujours pas une mise à jour de la position du robot ou un message confirmant sa position actuelle, il considérera alors qu'aucun robot n'est présent sur l'écran d'interaction.

V. Génération de trajectoire du robot

Le robot ayant un rôle clé dans l'exploration des DST, il est nécessaire qu'il puisse se déplacer en toute autonomie sans nécessairement nécessiter l'aide de l'utilisateur. En effet, lorsque l'utilisateur souhaite que le robot parcoure l'ensemble des données, il doit alors rejoindre les premiers points spatio-temporels des données pour pouvoir parcourir la suite, et cela par lui-même de manière à minimiser la charge de travail pour l'utilisateur.

Afin de permettre au robot depuis sa position de rejoindre n'importe quel emplacement sur l'écran d'interaction, un algorithme de génération de trajectoire fut développé. Cet algorithme se divise en plusieurs étapes (voir Figure 12) :

- Étape 1 : détecter la position P_r et l'orientation Θ_r actuelles du robot, ainsi que la position P_f et l'orientation Θ_f que le robot doit chercher à atteindre.
- Étape 2 : déterminer le centre d'un cercle O_r de rayon R dont l'un des points est tangent au vecteur directeur sous-jacent de l'orientation du point que le robot doit atteindre. Le robot va suivre un arc de ce cercle pour atteindre la position et l'orientation voulue.

- Étape 3 : de la même manière, on crée un cercle de centre P_r autour du robot de même rayon R que celui de l'étape 2. Le robot suivra également un arc de ce cercle pour pouvoir s'orienter vers le cercle de centre O_r .
- Étape 4 : déterminer les deux points A et B de chacun de ces cercles dont la tangente est commune aux deux cercles, tel que A (respectivement B) appartenant au premier cercle et A' (respectivement B') appartenant au second cercle partage une droite qui est tangente aux deux cercles.
- Étape 5 : en fonction de l'orientation finale Θ_f à atteindre, déterminer le sens de rotation que le robot devra suivre pour arriver à la position finale selon l'orientation souhaitée. Si $-\pi/2 < \Theta_f < \pi/2$, alors le robot parcourra les arcs de cercle selon le sens anti-trigonométrique, sinon il les parcourra dans l'autre sens.
- Étape 6 : éliminer les points A et A' ou B et B' en fonction du sens de rotation calculé.
- Étape 7 : tracer les arcs de cercle, le segment permettant au robot de rejoindre le cercle qui l'entoure et enfin le segment AA' ou BB' selon le sens de rotation choisi.

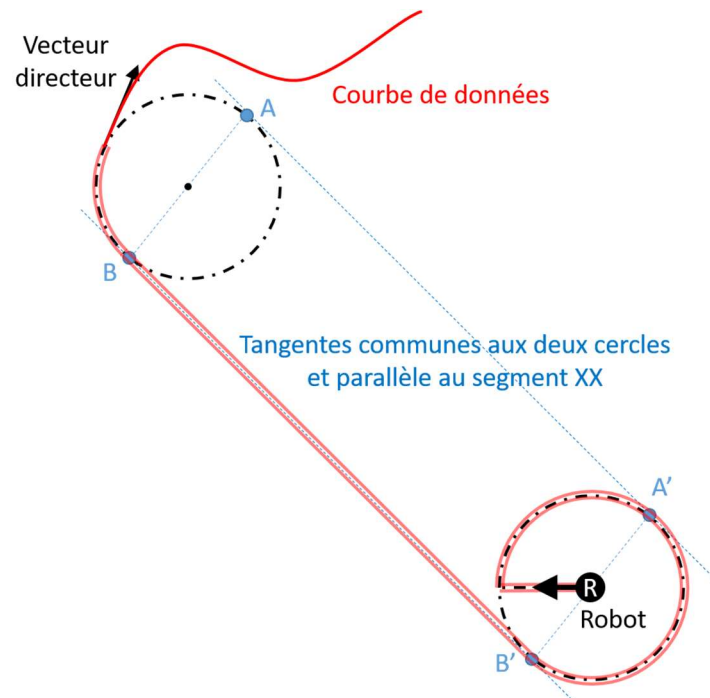


Figure 12 : Schéma descriptif de l'algorithme de génération de trajectoire circulaire

Plusieurs optimisations ont été imaginées. La première, la plus intuitive, est tout simplement de ne pas créer un segment entre le robot et son cercle de rotation, mais de placer ce cercle de manière à ce que le robot soit placé directement tangent sur son cercle de rotation (voir Figure 13).

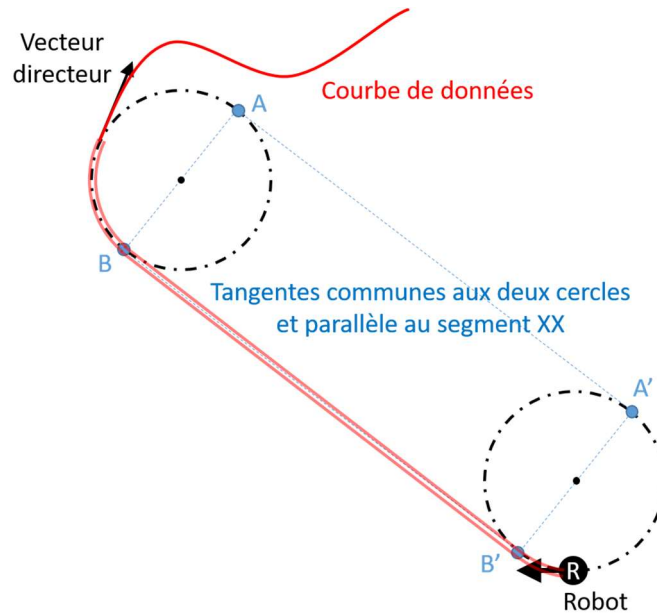


Figure 13 : Placement du robot tangent à son cercle de rotation

La seconde concerne l'optimisation de trajectoire en termes de distance. En effet, il est possible de minimiser la distance parcourue par le robot pour atteindre la position et orientation finale en choisissant de manière optimale le centre du cercle Or (voir Figure 14). Actuellement, celui-ci est défini selon le vecteur perpendiculaire à l'orientation finale Θ_f dans le sens horaire. Dans certains cas, il serait préférable de choisir ce vecteur perpendiculaire dans le sens trigonométrique.

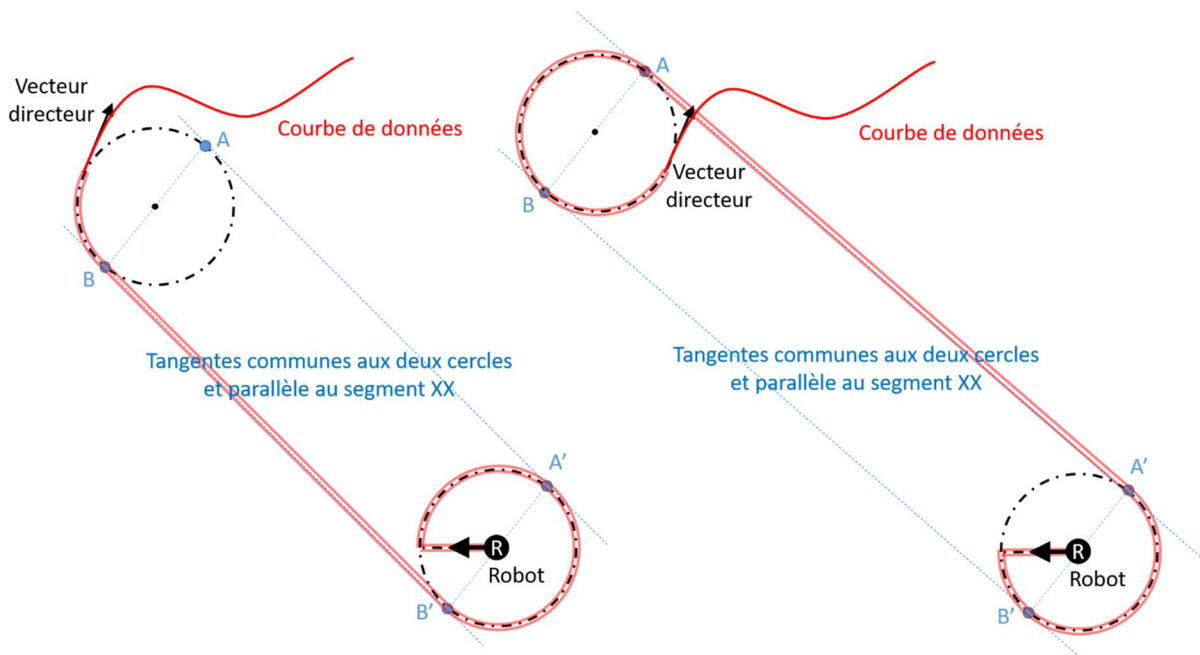


Figure 14 : Optimisation de trajectoire selon la distance à parcourir

VI. Création de l'interface graphique à partir du cube spatio-temporel

De manière à interagir et à visualiser des informations du CST, une interface graphique fut mise en place. Il est difficile de se représenter des données en 3D, cela demandant un effort cognitif important. Afin de faciliter ce travail, nous avons proposé de placer virtuellement trois caméras

virtuelles offrant une vue de trois plans de visualisation différents de ce CST : le plan XY, le plan XT et le plan TY. De cette façon, il est possible à l'utilisateur d'identifier chaque point de la courbe 3D de l'itinéraire du cycliste rapidement et efficacement. Bien que pratique, cette méthode demande une place importante sur la surface de l'interface graphique. Par conséquent, nous avons choisi de minimiser la vue de ces trois plans de visualisation et de créer un plan de coupe selon le plan de visualisation XY, appelé le plan de coupe principal. L'utilisateur pourra alors parcourir temporellement le cube via ce plan de coupe principal (à l'aide des flèches rouges en haut à droite de l'interface graphique), et donc étudier la position du cycliste à chaque instant. En utilisant cette méthode, l'utilisateur peut dans certains cas ne pas connaître la position du plan de coupe principal au sein du CST. Par conséquent, à tout instant, ce plan de coupe principal est visible sur chacun des plans de visualisation. Ce plan de coupe principal est non seulement dédié à la visualisation des DST selon un instant donné ou un intervalle de temps donné, mais est également une zone d'interaction pour le robot. C'est sur ce plan de coupe principal que le robot va pouvoir se déplacer, mettre en évidence des informations ou représenter l'entité étudiée au travers de ces données.

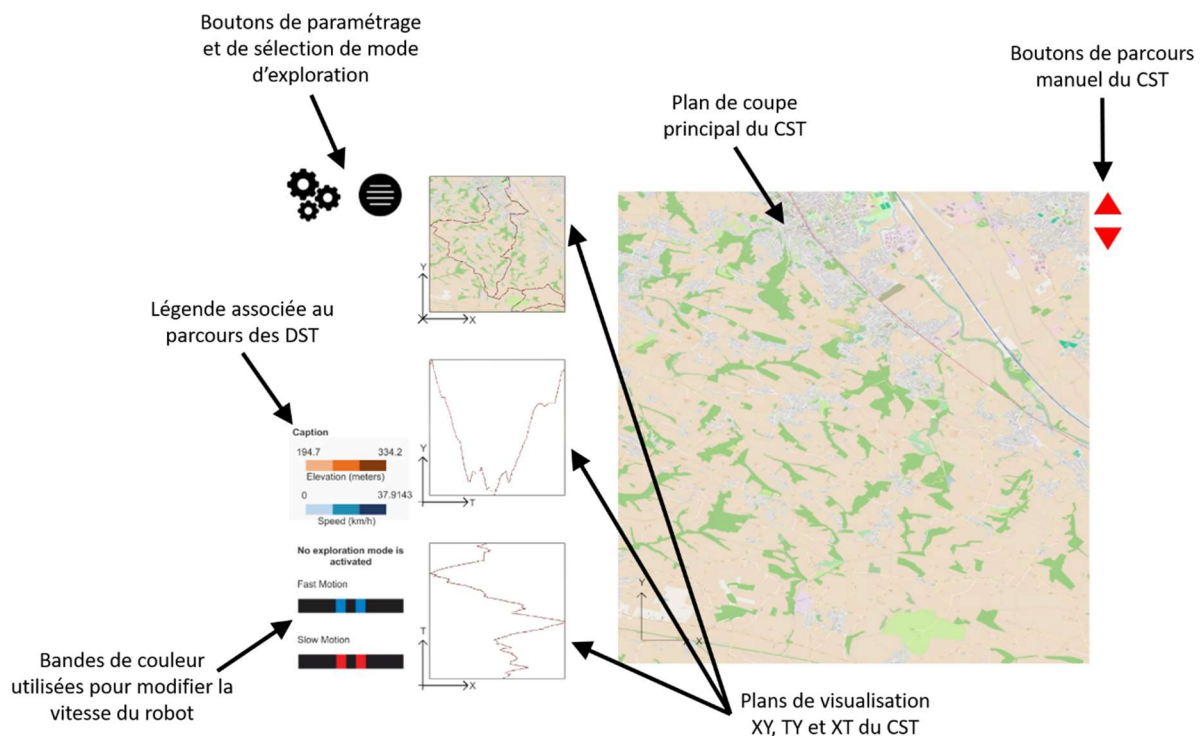


Figure 15 : Capture de l'interface graphique du système

En supplément de ces plans de visualisation et de ce plan de coupe principal, d'autres éléments sont visibles sur cette interface graphique (voir Figure 15). En effet, on peut observer 4 zones différentes. Une zone en haut à gauche présente deux boutons utilisés pour le paramétrage de l'application et le second utilisé pour la sélection du mode d'exploration désiré. En dessous, on retrouve une zone appelée « Caption ». Cette zone représente les légendes de couleur qui apparaîtront autour du robot. Puis, on retrouve une zone de texte indiquant le mode d'exploration choisi par l'utilisateur. Enfin, des bandes de couleurs sont visibles en bas à gauche de l'interface graphique. Elles ont pour rôle de modifier la vitesse de déplacement du robot.

1. Boutons de paramétrage et de sélection du mode d'exploration

Les enjeux de ce projet furent de développer une application capable d'utiliser des concepts de visualisation et d'exploration des DST innovantes, mais également de pouvoir adapter ce prototype à différentes plateformes matérielles. En effet, cette application est vouée à être déportée sur une table interactive tactile. Par conséquent, de nombreux paramètres furent mis en place de manière à pouvoir s'adapter à n'importe quelle taille d'écran et à réagir à des événements clics souris ou tactiles.

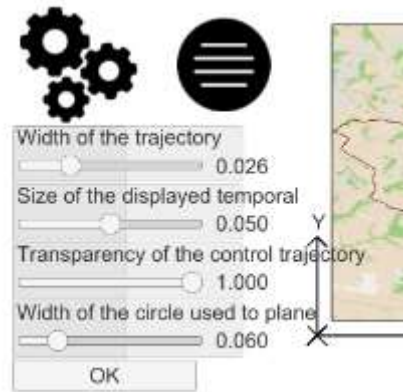


Figure 16 : Paramétrage du programme d'exploration des DST via l'interface graphique

Ces paramètres sont accessibles via le bouton en haut à gauche de la forme d'un écrou. Lors d'un clic souris ou tactile sur ce bouton, un menu apparaît avec différentes variables paramétrables (voir Figure 16). Parmi ces paramètres, on retrouve :

- la largeur de la trajectoire qui peut varier en fonction de la taille de l'écran en sachant qu'elle doit au minimum avoir 1 cm de largeur sur l'écran d'interaction pour que le robot Ozobot puisse la suivre sans encombre,
- l'intervalle temporel de visualisation en fonction de si l'utilisateur veut explorer les données minute par minute, heure par heure, jour par jour, etc.,
- l'opacité du cercle blanc autour du robot pour l'empêcher de suivre un autre objet que la courbe de données ou la trajectoire affichée pour la rejoindre (voir la section « III. Choix du type de robot utilisé » de la section « Description technique des concepts, des outils logiciels et matériels »),
- la largeur du rayon du cercle de rotation du robot et du cercle de rotation tangent aux premiers points de la courbe de données (voir la sous-section « V. Génération de trajectoire du robot » de la section « Description technique des concepts, des outils logiciels et matériels »).

Plusieurs modes d'exploration des données sont proposés à l'utilisateur pour accéder aux informations contenues dans les DST étudiées. Ces modes d'explorations des données seront expliqués plus en détail par la suite (voir la section « Mode d'exploration des DST »). Leur sélection toutefois ne nécessite pas leur compréhension.



Figure 17 : Sélection du mode d'exploration des données via l'interface graphique du programme d'exploration des DST

En effet, pour sélectionner un mode d'exploration des données, l'utilisateur peut cliquer sur le bouton à droite du bouton de paramétrage pour faire apparaître un menu présentant les différents modes d'exploration disponibles (voir Figure 17). Il est également possible pour l'utilisateur de désactiver tout mode d'exploration en cliquant sur la dernière proposition du menu s'il le souhaite. Dans ce cas, le programme fera disparaître les différents éléments liés aux DST qui apparaissaient sur la coupe principale du CST. En revanche, les informations visibles depuis les trois plans de visualisations resteront disponibles et ne disparaîtront pas, procurant une vue globale des DST à tout moment.

2. Légendes de couleur autour du robot

Lorsque le robot parcourt les DST, l'utilisateur a accès non seulement à ses coordonnées spatiales X et Y à un instant T, mais également à des informations telles que l'élévation et la vitesse instantanée de l'entité étudiée à ce même instant T. Ces informations sont disponibles autour du robot sous la forme de cercles de couleurs et d'informations textuelles (voir Figure 18).



Figure 18 : Photo du parcours des DST par le robot Ozobot avec l'inclusion d'informations complémentaires

Le cercle de couleur orangée représente les informations liées à l'élévation du point spatio-temporel le plus proche du robot. Le cercle de couleur bleutée représente les informations liées à la vitesse instantanée. Toutefois, la vitesse instantanée n'est pas directement contenue dans le fichier

GPX. Elle est calculée à partir des données GPS capturées lors de l'enregistrement du suivi de l'entité étudiée.

Pour calculer la vitesse instantanée entre deux points dont les coordonnées GPS (latitude1, longitude1, latitude2, longitude2) et l'intervalle de temps entre ces deux points spatio-temporels sont connus, on réalise une série de calculs prenant compte le fait que la Terre n'est pas une sphère parfaite [2] :

- Étape 1 : déterminer la latitude moyenne « $latitude_{moy}$ » entre les deux points spatio-temporels : $latitude_{moy} = \frac{(latitude_1 + latitude_2)}{2}$
- Étape 2 : déterminer la distance en mètre entre chaque degré de latitude « $distance_{lat}$ » :
 $distance_{lat} = 111132.94 - 559.822 \times \cos(2 \times latitude_{moy}) + 1.175 \times \cos(4 * latitude_{moy})$
- Étape 3 : déterminer la distance en mètre entre chaque degré de longitude « $distance_{long}$ » :
 $distance_{long} = 111412.84 \times \cos(latitude_{moy}) - 93.5 \times \cos(3 \times latitude_{moy})$
- Étape 4 : déterminer la distance en mètre entre ces deux points spatio-temporels « $distance$ » :
 $distance = \sqrt{(latitude_1 - latitude_2)^2 + (longitude_1 - longitude_2)^2}$
- Étape 5 : déterminer la vitesse de déplacement entre ces deux points spatio-temporels « $vitesse$ » en fonction de l'intervalle de temps Δt entre la capture de ces deux points :
 $vitesse = \frac{distance}{\Delta t}$

3. Modification de la vitesse du robot

Par défaut à son allumage, le robot adopte une vitesse de croisière de 3,3 centimètres par seconde. Il est possible de modifier cette vitesse en positionnant une bande de couleur selon un motif bien précis. Par exemple, lorsque le robot détecte séquentiellement une ligne de couleur bleue-noire-bleue, sa vitesse de déplacement devient plus lente avec un déplacement de 2,3 centimètres par seconde. Lorsqu'il détecte la séquence rouge-noir-rouge, sa vitesse de déplacement est alors plus rapide (5,3 centimètres par seconde).

Par le biais de ces bandes de couleurs en bas à gauche de l'interface graphique (voir Figure 15), il est alors possible d'explorer plus ou moins vite les DST. À tout moment, l'utilisateur peut saisir le robot et le placer sur l'une de ces lignes pour faire varier sa vitesse. S'il souhaite revenir à la vitesse de croisière originale, il peut alors appuyer deux fois sur le bouton situé sur la gauche du robot.

Modes d'exploration des DST

Comme mentionné précédemment, l'exploration des DST est cruciale pour comprendre les modèles mathématiques sous-jacents de ces jeux de données. C'est pourquoi nous proposons plusieurs modes d'exploration. Parmi ces différents modes, on retrouve les modes d'exploration suivants : « Manual Data Mining », « Automatic Data Mining », « Elevation-Related Interest Segment Selection » et « User-Defined Interest Segment Selection ». Chacun d'eux offre soit un point de vue d'analyse des DST différent, soit un accès à des informations supplémentaires. Quel que soit le mode d'exploration choisi par l'utilisateur, le robot représentera l'objet des DST (qui dans le scénario initial du parcours sportif représente le cycliste, ou un étudiant dans le cas du scénario néOCampus). Il pourra parcourir l'ensemble des DST mettant en évidence un nombre limité d'informations, ou il pourra être saisi par l'utilisateur et ainsi afficher des informations supplémentaires sur l'écran d'interaction.

Pour la description de l'ensemble des modes d'exploration implémentés, nous décrirons leur utilisation et leurs objectifs selon le scénario du suivi de l'entraînement du cycliste. Par la suite, nous décrirons les variantes apportées au programme pour s'adapter à un autre scénario, celui de l'étude énergétique des laboratoires partenaires de néOCampus, le but étant d'expliquer comment l'application peut s'adapter à plusieurs ensembles de données différents.

Quel que soit le mode d'exploration choisi par l'utilisateur, à la sélection de l'un de ces modes, le robot Ozobot utilisé pour ce premier prototype du programme d'exploration de DST se rendra par lui-même sur les premiers points spatio-temporels affichés ou sur le point d'intérêt sélectionné par l'utilisateur. En effet, lorsque l'utilisateur sélectionnera le mode d'exploration « Manual Data Mining » ou « Automatic Data Mining », une courbe noire représentant les DST apparaîtra en supplément d'une courbe rouge (générée selon l'algorithme de la sous-section « V. Génération de trajectoire du robot » de la section « Description des concepts, outils logiciels et matériels ») lui permettant de rejoindre les premiers points de cette courbe noire. De même, lors de la sélection d'un point d'intérêt parmi l'ensemble des DST, une courbe noire représentant les DST de ce point d'intérêt jusqu'au point d'intérêt suivant apparaîtra en supplément d'une courbe rouge permettant au robot de rejoindre les premiers points de cette courbe noire.

I. « Manual Data Mining »

Ce mode d'exploration est le plus simple et intuitif de tous ceux proposés dans l'application. En effet dans ce cas-ci, l'utilisateur peut explorer le CST par tranche temporelle. Une tranche temporelle représente soit une coupe du cube à un instant donné, soit une coupe du cube sur un intervalle de temps donné. Par exemple, l'utilisateur peut explorer chacune des coupes instantanées du CST une à une, c'est-à-dire l'intégralité des DST point par point, ou alors il peut explorer les DST minute par minute, heure par heure ou jour par jour en fonction de l'intervalle de temps choisi (voir la sous-section « VI. Création de l'interface graphique » de la section « Description des concepts, des outils logiciels et matériels »).

Au démarrage de ce mode, le robot va rejoindre le premier point spatio-temporel de l'ensemble des DST extraites du fichier GPX via une courbe de couleur rouge générée selon l'algorithme décrit précédemment (voir la sous-section « VI. Génération de trajectoire du robot » de la section « Description des concepts, des outils logiciels et matériels »). Dès que le robot atteindra la courbe de

données affichée en noire, la courbe rouge disparaîtra et deux petits cercles de couleur bleutée et orangée apparaîtront. Le cercle de couleur orangée contiendra les informations liées à l'élévation, et celui de couleur bleutée contiendra les informations liées à la vitesse instantanée du cycliste calculée depuis les informations spatiales contenues dans les DST (voir la sous-section « VI. Création de l'interface graphique » de la section « Description des concepts, des outils logiciels et matériels »).

Le robot Ozobot étant un suiveur de ligne, il suivra alors la courbe noire jusqu'à son dernier point affiché. Le nombre de points contenus dans cette courbe 3D est dépendant de l'intervalle de temps défini par défaut ou par l'utilisateur. En effet, sans changement de l'utilisateur, le système utilisera un intervalle temporel égal au centième du temps total des DST. Par exemple, si l'ensemble des DST est réparti sur 100 jours, alors le système affichera les DST selon un intervalle temporel d'un jour. À tout moment, ce paramètre est modifiable dans les paramètres de l'application (voir la sous-section « VI. Création de l'interface graphique » de la section « Description des concepts, des outils logiciels et matériels »).

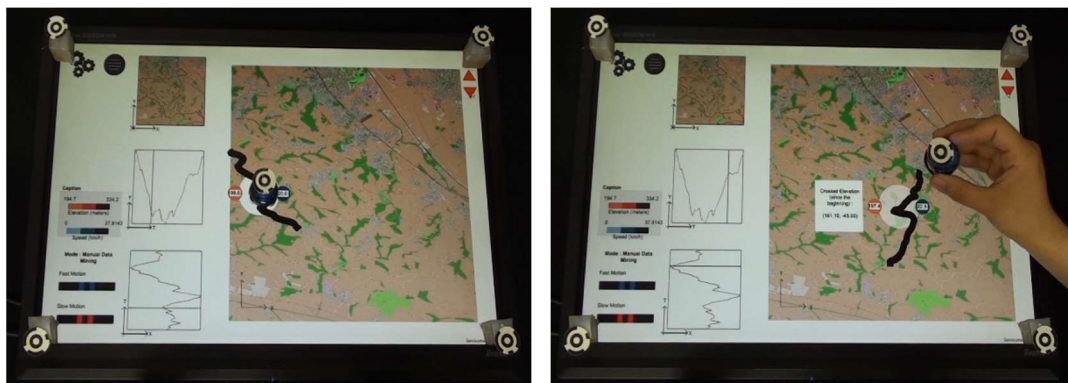


Figure 19 : Photos d'une démonstration du mode d'exploration « Manual Data Mining »

Ce mode d'exploration permet à l'utilisateur d'explorer en détail chacun des points spatio-temporels. Toutefois, en laissant le robot se déplacer par lui-même sur cette courbe représentant un sous-ensemble des DST, il est possible que l'utilisateur ne puisse pas accéder à toutes les informations qu'il souhaiterait. Nous proposons donc une autre manière d'interagir avec le système, en utilisant le robot comme interface tangible. L'utilisateur aura alors accès à des informations liées non pas à un seul point, mais à l'ensemble des DST déjà parcouru ou restant à parcourir, lui permettant d'avoir une vue d'ensemble des informations contenues dans les DST. On définit alors trois paliers d'élévation du robot : le « Palier 1 » lorsque le robot est soulevé entre 2 et 10 centimètres au-dessus de l'écran d'interaction, « Palier 2 » lorsqu'il est soulevé entre 10 et 20 centimètres et le « Palier 3 » lorsqu'il est soulevé à plus de 20 centimètres. En soulevant le robot sur trois paliers d'élévation, l'utilisateur pourra visualiser des informations relatives à l'élévation parcourue ou à parcourir depuis la position où le robot a été soulevé. Il pourra également visualiser les informations relatives à la vitesse moyenne du cycliste depuis le début de l'entraînement ou celle qui l'a adopté de la position du robot jusqu'au dernier point spatio-temporel enregistré (voir Tableau 1).

	Informations affichées lorsque $0^\circ < \Theta_{\text{robot}} < 180^\circ$	Informations affichées lorsque $180^\circ < \Theta_{\text{robot}} < 360^\circ$
Palier 1 d'élévation (entre 2 et 10 cm d'altitude)	Carré blanc avec une photo du point d'intérêt le plus proche s'il y en a un à proximité	
Palier 2 d'élévation (entre 10 et 20 cm d'altitude)	Élévation géographique parcourue depuis le début de l'entraînement	Élévation géographique restant à parcourir jusqu'au dernier point spatio-temporel des DST
Palier 3 d'élévation (entre 20 et 30 cm d'altitude)	Vitesse moyenne depuis le début de l'entraînement	Vitesse moyenne estimée jusqu'au dernier point spatio- temporel des DST

Tableau 1 : Récapitulatif des informations affichées lors de la saisie du robot
au cours du mode d'exploration « Manual Data Mining »

Dans le cas où l'utilisateur souhaite parcourir manuellement la courbe de données affichées (sans que le robot la parcoure par lui-même), il peut faire glisser le robot le long de cette courbe pour étudier la vitesse instantanée et l'élévation instantanée de chacun des points de cette courbe. Utilisant cette méthode, il peut aussi placer avec précision le robot au-dessus du point qu'il souhaite prendre comme référence pour étudier l'élévation parcourue ou restante à parcourir, ou encore la vitesse moyenne depuis le début de l'entraînement, et ce jusqu'à la fin.

II. « Automatic Data Mining »

Ce mode d'exploration est très similaire au précédent. De la même manière, au démarrage de ce mode, le robot rejoindra les premiers points de la courbe de données via une courbe rouge. Une fois que le robot aura atteint la courbe noire, la courbe de couleur rouge disparaîtra et les cercles de couleur indiquant l'élévation instantanée et la vitesse instantanée apparaîtront autour du robot. L'intervalle temporel de l'application est encore une fois utilisé pour définir le nombre de points spatio-temporels que contiendra la courbe noire. La différence par rapport au mode précédent est que l'utilisateur n'a pas besoin d'appuyer sur les boutons en haut à droite de l'interface graphique pour avancer temporellement dans le CST. La courbe s'adaptera d'elle-même autour du robot de manière à ce que celui-ci soit toujours au centre de la courbe.

Dans le document annexe « État de l'art sur l'exploration de données spatio-temporelles », l'utilisation d'animations pour communiquer des informations spatio-temporelles est largement étudiée dans la communauté scientifique. Dans notre cas, le but est de communiquer les performances sportives à travers le mouvement du robot. Étant difficile de changer sa vitesse dynamiquement, cette fonctionnalité fut tout de même mise en place dans le cas où un changement de robot aurait lieu. En effet, s'il est possible de faire varier la vitesse du robot dynamiquement, il serait alors possible de communiquer la vitesse instantanée du cycliste lors de son parcours par la vitesse de déplacement du robot lors de son parcours des DST. Actuellement, bien que le robot Ozobot ne puisse pas modifier sa vitesse de manière dynamique, il est tout de même capable de mettre en évidence les zones où le cycliste s'est arrêté pour se reposer.

De même que pour le mode « Manual Data Mining », l'utilisateur pourra saisir le robot pour parcourir manuellement la courbe de données affichée, ou pour accéder à des données supplémentaires (comme l'élévation parcourue ou à parcourir, ou encore la vitesse moyenne du parcours) depuis un point de référence choisi (voir Tableau 1).

III. « Elevation-Related Interest Segment Selection »

À l'aide des modes « Manual Data Mining » et « Automatic Data Mining », il est possible d'étudier la vitesse du cycliste et son élévation géographique à chaque instant. Cela nous permet de faire des suppositions sur les conséquences d'un ralentissement ou d'une accélération au cours de son entraînement, comme par exemple un ralentissement au cours d'une montée ou encore une accélération au cours d'une descente. Seulement, il est difficile de déterminer en quel point exactement cette situation a pu changer avec les modes précédents. Un nouveau mode d'exploration fut alors créé prenant compte des différentes variations d'élévation positives et négatives sur le parcours du cycliste. Ce mode permet de mettre en évidence les pics de descente ou de montée expliquant les variations de vitesse de déplacement du cycliste (voir Figure 20).



Figure 20 : Photos d'une démonstration du mode d'exploration « Elevation-Related Interest Segment Selection »

Au démarrage de ce mode, l'ensemble des points d'intérêts liés à l'élévation sont affichés sur le plan de coupe principal. De plus, un menu déroulant cliquable regroupant l'ensemble des points d'intérêt affichés sur ce plan de coupe principal est disponible en haut à gauche de l'interface graphique pour que l'utilisateur puisse sélectionner un ou plusieurs de ces points. S'il ne souhaite pas utiliser ce menu déroulant, il peut directement cliquer sur les petits cercles apparus sur le plan de coupe principal. Les deux méthodes de sélection donneront le même résultat. On remarque également que des courbes très fines relient chaque point d'intérêt. Ces courbes représentent l'intégralité des DST selon une grande courbe spatiale passant par chacun des points spatio-temporels référencés dans le fichier GPX dont les données furent extraites. Ainsi l'utilisateur peut visualiser l'ensemble des données et les points d'intérêt parmi ces données en une seule et même visualisation.

Lorsque l'utilisateur sélectionne un point d'intérêts liés à l'élévation selon l'une des deux méthodes énoncées, la courbe présente entre le point sélectionné et le point d'intérêt suivant sera alors épaissie afin que le robot soit en mesure de la suivre. D'autre part, de la même manière que pour les autres modes, une courbe rouge apparaîtra afin de permettre au robot de rejoindre le point d'intérêt sélectionné et selon l'orientation du vecteur directeur des premiers points de la courbe noire représentant les DST. En sélectionnant les points d'intérêt manuellement, l'utilisateur pourra parcourir l'ensemble des données en fonction de la variation de l'élévation au cours du parcours du cycliste. Il pourra donc étudier les performances du cycliste en fonction de la longueur de la montée ou de la descente, de la pente ou encore, il pourra comparer les performances entre les différents segments étudiés lui permettant ainsi d'évaluer si les performances du cycliste s'améliorent au cours du temps ou se dégradent. Afin de lui permettre d'évaluer tous ces critères, lorsque le robot est saisi par l'utilisateur sur l'un de ces segments d'intérêt, de nouvelles informations sont affichées. Ces

informations sont différentes des autres modes d'exploration et sont relatives au segment que l'utilisateur est en train d'explorer (voir Tableau 2).

	Informations affichées lorsque $0^\circ < \Theta_{\text{robot}} < 180^\circ$	Informations affichées lorsque $180^\circ < \Theta_{\text{robot}} < 360^\circ$
Palier 1 d'élévation (entre 2 et 10 cm d'altitude)	Carré blanc avec une photo du point d'intérêt le plus proche s'il y en a un à proximité	
Palier 2 d'élévation (entre 10 et 20 cm d'altitude)	Longueur en mètres du segment étudié	Variation d'élévation en mètre le long du segment étudié
Palier 3 d'élévation (entre 20 et 30 cm d'altitude)	Nombre de fois où le cycliste a parcouru ce segment	Nombre de cyclistes ayant déjà parcouru ce segment

Tableau 2 : Récapitulatif des informations affichées lors de la saisie du robot
au cours du mode d'exploration « Elevation-Related Interest Segment Selection »

De la même manière que pour tous les autres modes, l'utilisateur pourra saisir le robot pour parcourir manuellement le segment d'intérêt affiché ou déplacer le robot avec précision au-dessus des points du segment dans le but de le soulever ensuite pour obtenir des informations supplémentaires (voir Tableau 2).

IV. « User-Defined Interest Segment Selection »

Ce mode d'exploration est semblable au mode d'exploration précédent « Elevation-Related Interest Segment Selection », à la différence que les points d'intérêt sont définis à l'avance avec l'utilisateur. Par exemple, dans le cas de ce scénario du parcours du cycliste, les points intérêt représentent le centre des villes par lesquelles le cycliste est passé lors de son entraînement. L'inconvénient de cette méthode est qu'il est nécessaire de définir à l'avance les points d'intérêt avec l'utilisateur en fonction de ses besoins. Actuellement, ces points intérêt sont créés statiquement dans le code source de l'application, mais pourront ensuite être rentrés manuellement depuis l'interface graphique ou à l'aide d'un fichier texte externe. Pour chaque point d'intérêt créé, une photo ou un logo lui est associé. Quel que soit le mode d'exploration choisie par l'utilisateur, à chaque fois que le robot passera à proximité de l'un de ses points d'intérêts, la photo ou le logo associé au point d'intérêt le plus proche apparaîtra sur la coupe principale du CST. En effet, nous avons considéré que si l'utilisateur entre manuellement ces points d'intérêts dans le programme, cela signifie qu'ils ont une importance pour lui. Donc, si cela ne perturbe pas le robot ou la visualisation des données qu'il souhaite explorer, l'ajout des logos ou des photos indiquant l'emplacement des points d'intérêt (quel que soit le mode) peut apporter une personnalisation de l'exploration des données.

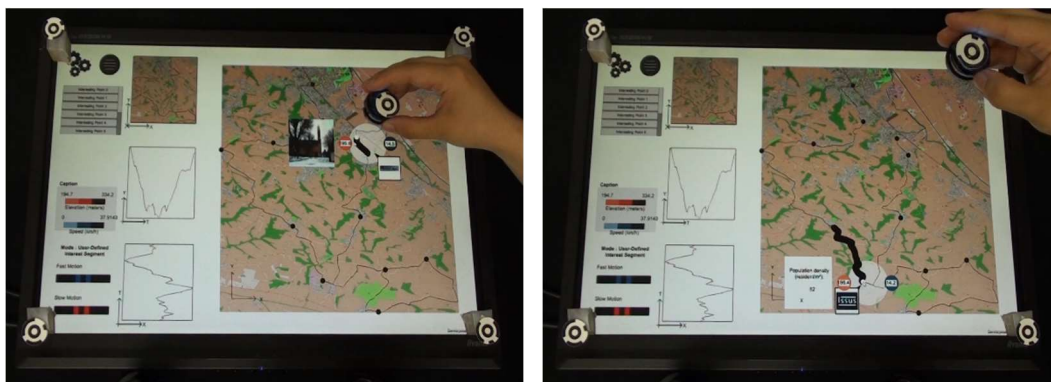


Figure 21 : Photos d'une démonstration du mode d'exploration « User-Defined Interest Segment Selection »

À la sélection de ce mode d'exploration, un menu déroulant cliquable avec l'ensemble des points d'intérêts répertoriés sera visible en haut à gauche de l'interface graphique et des petits cercles représentant les points d'intérêt apparaîtront sur la coupe principale du CST. On retrouvera encore une fois une fine courbe entre chaque point intérêt correspondant à l'itinéraire complet qu'aura suivi le cycliste lors de l'enregistrement des données. À la sélection d'un point d'intérêt, une courbe noire plus épaisse apparaîtra depuis le point d'intérêt sélectionné jusqu'au point intérêt suivant, ainsi qu'une courbe rouge permettant au robot de rejoindre le point d'intérêt sélectionné. Comme pour les autres modes, une fois que le robot aura atteint la fin de la courbe rouge, celle-ci disparaîtra et les cercles indiquant la vitesse et l'élévation instantanées du point spatio-temporel le plus proche du robot apparaîtront. L'utilisateur pourra donc parcourir les DST de ville en ville, lui permettant non pas d'étudier cette fois les performances sportives du cycliste, mais d'étudier le parcours spatial qu'il a emprunté en obtenant par exemple des informations sur les villes que le cycliste a traversées. À tout moment, il pourra saisir le robot pour avoir accès à ces informations (voir Tableau 3).

	Informations affichées lorsque $0^\circ < \theta_{\text{robot}} < 180^\circ$	Informations affichées lorsque $180^\circ < \theta_{\text{robot}} < 360^\circ$
Palier 1 d'élévation (entre 2 et 10 cm d'altitude)	Carré blanc avec une photo du point d'intérêt le plus proche s'il y en a un à proximité	
Palier 2 d'élévation (entre 10 et 20 cm d'altitude)	Total d'habitants recensés dans la ville étudiée	Âge moyen des habitants de la ville
Palier 3 d'élévation (entre 20 et 30 cm d'altitude)	Surface totale de la ville en m^2	Densité de la population en habitant/ m^2

Tableau 3 : Récapitulatif des informations affichées lors de la saisie du robot
au cours du mode d'exploration « User-Defined Interest Segment Selection » (scénario cycliste)

Les informations contenues dans le Tableau 3 furent extraites des sites Internet « drimki.fr » et « wikipedia.org ». Bien qu'imprécises, elles suffisent à démontrer l'utilité d'un tel mode d'exploration.

Comme pour tous les autres modes, l'utilisateur pourra saisir le robot pour parcourir manuellement le segment d'intérêt affiché ou déplacer le robot avec précision au-dessus des points spatio-temporels du segment dans le but de le soulever pour obtenir des informations supplémentaires (voir Tableau 3).

V. Adaptation du programme d'exploration pour le scénario néoCampus

Dans ce scénario, nous suivons un étudiant se promenant sur le campus universitaire de l'université Paul Sabatier III et passant par les différents laboratoires partenaires du projet néoCampus. Globalement, peu de changements par rapport au scénario du cycliste furent apportés pour que l'application puisse s'adapter aux nouvelles DST. Parmi les changements effectués, on retrouve :

- un nouveau fichier GPX contenant les DST de l'itinéraire de l'étudiant sur le campus universitaire,
- une nouvelle carte OpenStreetMap téléchargée depuis le logiciel Taho et chargée dans l'application,
- une nouvelle liste de points d'intérêt définis par l'utilisateur et insérée dans le programme pour le mode d'exploration « User-Defined Interest Segment Selection » et le placement des logos et photos des points d'intérêt,

- une nouvelle liste d'informations disponibles par interaction tangible avec le robot pour le mode d'exploration « User-Defined Interest Segment Selection ».

Le projet néOCampus regroupe 11 laboratoires partenaires combinant leurs compétences dans différents domaines (informatique, matériaux, électricité, écologie, etc.) pour construire le campus universitaire de demain : intelligent, innovant et durable. Dans ce scénario, il n'y a donc pas de sens d'explorer les DST de ville en ville. En effet, les données sont regroupées sur une zone spatiale très limitée (quelques kilomètres de diamètre). Cependant, au vu des objectifs du projet néOCampus, il est intéressant de pouvoir étudier les dépenses énergétiques des bâtiments.

Dans ce scénario, nous définissons manuellement 11 points d'intérêt représentant les 11 laboratoires partenaires du projet néOCampus. Dans le mode d'exploration « User-Defined Interest Segment Selection », il sera possible d'interagir physiquement avec le robot pour obtenir des informations sur la consommation énergétique de ces 11 laboratoires (voir Tableau 4).

	Informations affichées lorsque $0^\circ < \theta_{\text{robot}} < 180^\circ$	Informations affichées lorsque $180^\circ < \theta_{\text{robot}} < 360^\circ$
Palier 1 d'élévation (entre 2 et 10 cm d'altitude)	Carré blanc avec une photo du point d'intérêt le plus proche s'il y en a un à proximité	
Palier 2 d'élévation (entre 10 et 20 cm d'altitude)	Consommation en électricité instantanée du laboratoire étudié	Consommation en électricité annuelle du laboratoire étudié
Palier 3 d'élévation (entre 20 et 30 cm d'altitude)	Consommation en eau instantanée du laboratoire étudié	Consommation en eau annuelle du laboratoire étudié

Tableau 4 : Récapitulatif des informations affichées lors de la saisie du robot
au cours du mode d'exploration « User-Defined Interest Segment Selection » (scénario néOCampus)

Une partie de ces informations furent extraites de relevés énergétiques procurés par l'IIRIT. Pour les laboratoires dont ces informations sont confidentielles, une petite variation aléatoire des chiffres extraits des relevés précédents fut insérée pour garder les consommations énergétiques cohérentes à d'autres bâtiments de taille et de fonction similaires.

Cette adaptation du projet fut présentée lors de la journée du projet néOCampus sous la forme d'une présentation orale ainsi que d'un stand de présentation muni d'un poster et de vidéos de démonstration de l'application.

Code source, exécutable et documentation

Les enjeux de ce premier prototype furent de développer une application capable d'utiliser des concepts innovants de visualisation et d'exploration des DST, mais également de pouvoir adapter ce prototype à différentes plateformes matérielles et logicielles. Pour plus de facilité d'utilisation, des fichiers exécutables (pour chacun des deux scénarios) du programme furent créés en 64 bits et 32 bits compatibles avec des systèmes d'exploitation tels que Linux et Windows. Ces fichiers exécutables furent générés depuis Eclipse pour le programme de localisation du robot et depuis Unity à l'aide de Visual Studio 2017 pour le programme principal incluant l'extraction des DST et leur exploration via le concept du CST. Toutefois, cela n'est pas suffisant. Certaines installations et configurations sont nécessaires au préalable pour pouvoir lancer ces fichiers exécutables.

I. Installations et configurations

Dans un premier temps, il est nécessaire d'installer Java en 64 bits ou 32 bits sur l'ordinateur hôte. Ce programme est disponible à l'adresse suivante : <https://www.java.com/fr/>. En effet, le programme de détection étant en programme JAVA compressé sous une archive JAR, il est nécessaire d'installer ce programme pour que la commande système de lancement soit reconnue par le système d'exploitation.

D'autre part, pour que ce programme de localisation puisse fonctionner dans les meilleures conditions, l'utilisateur doit s'assurer qu'une caméra soit branchée et disponible sur son ordinateur. Si ce n'est pas le cas, le programme de localisation ne trouvera pas de périphérique vidéo et se terminera sans lancer son serveur local. Par conséquent, Unity ne pourra pas connaître la localisation du robot durant cette séance d'exploration des données.

Pour les systèmes d'exploitation tels que Windows, il est important de vérifier que le pare-feu de bloquera pas le lancement du serveur local utilisé comme intermédiaire de communication entre le programme de localisation et le programme d'exploration des données. En temps normal, Windows demandera au lancement du programme les autorisations nécessaires (voir Figure 22).

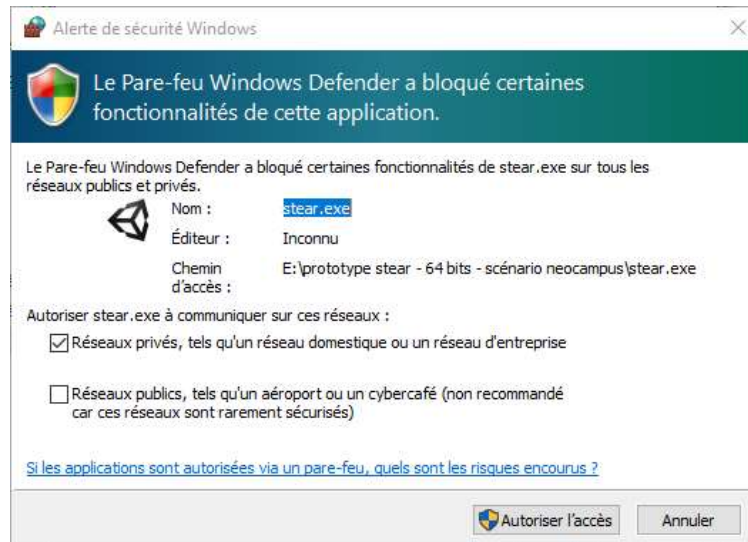


Figure 22 : Demande d'autorisation de Windows pour permettre à l'application de passer le pare-feu présent sur l'ordinateur hôte

Enfin, lorsque l'utilisateur lancera l'exécutable Unity, une fenêtre apparaîtra demandant certaines informations comme la résolution de l'application, l'affichage en plein écran ou fenêtré, etc. (voir Figure 23). L'utilisateur devra choisir la résolution 1600 x 1024 en mode plein écran ou fenêtrée pour que l'affichage de l'interface graphique reste cohérent. Si jamais l'utilisateur se trompe lors de la sélection de la résolution, l'application fonctionnera tout de même, mais les différents objets de l'interface graphique pourront être placés différemment.

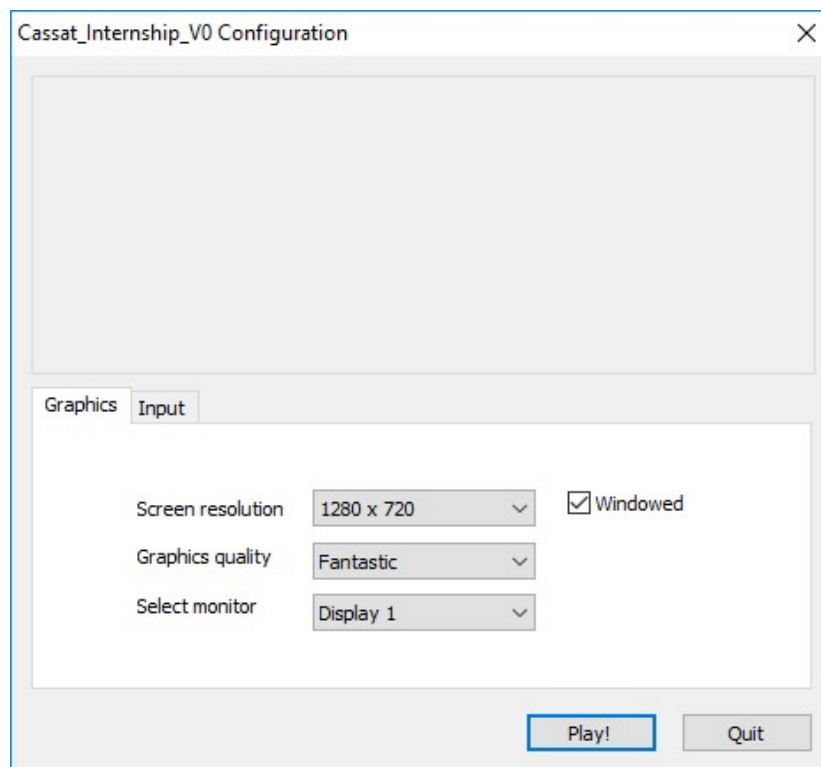


Figure 23 : Capture d'écran de la fenêtre de lancement du programme d'exploration des DST

II. Lancement de l'application développée

Pour lancer l'application, il suffit que l'utilisateur exécute le fichier « STEAR.exe » contenu dans le répertoire de son choix :

- Prototype STEAR - 32 bits - scénario cycliste,
- Prototype STEAR - 32 bits - scénario néOCampus,
- Prototype STEAR - 64 bits - scénario cycliste,
- Prototype STEAR - 64 bits - scénario néOCampus.

Lorsqu'il aura lancé le fichier exécutable, le programme principal développé sous Unity passera par les étapes suivantes :

- 1) Extraction des DST depuis le fichier GPX,
- 2) Création des objets 3D de l'application (CST, interface graphique, etc.),
- 3) Lancement du serveur de localisation du robot via une commande système,
- 4) Demande d'accès en boucle au serveur de localisation jusqu'à ce que celui-ci accepte,
- 5) Affichage de l'interface graphique et des objets 3D.

Une fois toutes ces étapes réalisées, l'utilisateur pourra alors utiliser les différents modes d'exploration de l'application pour visualiser, accéder et manipuler les informations contenues dans les DST extraites du fichier GPX.

III. Documentation du code source

Le projet étant voué à être réutilisé dans le futur, il est important de documenter le travail réalisé durant ce stage. Pour documenter le code source du projet, l'outil nommé Doxygen fut utilisé comme aide pour générer un manuel de toutes les classes, fonctions et commentaires importants à l'intérieur du code source de l'application d'exploration des DST (voir Figure 24). Plusieurs formats et options étaient disponibles pour générer ce document. Nous avons choisi de créer des documents HTML pour enregistrer toute la documentation en raison de sa facilité d'utilisation et de son format facilement interprétable par la plupart des navigateurs internet.

STEAR

Exploration de données spatio-temporelles

Main Page | Classes | Files | Search

Public Member Functions | Public Attributes | Protected Attributes | Private Member Functions | Private Attributes | List of all members

GPXParser Class Reference

! The GPXParser class extracts space-time data (spatial position, elevation, speed, ...) from a GPX file and adjusts its value to contain them within the space-time cube. More...

Inheritance diagram for GPXParser:

```

graph BT
    MonoBehaviour --|> GPXParser
  
```

Public Member Functions

ArrayList	getRealData ()	! Return the ArrayList<Vector3> attribut that represents the extracted spatio-temporal data from the GPX file. More...
ArrayList	getRealElevation ()	! Return the ArrayList<float> attribut that represents the extracted elevation data from the GPX file. More...
ArrayList	getAdjustedData ()	! Return the ArrayList<Vector3> attribut that represents the extracted spatio-temporal data from the GPX file, ajusted to fit within the space-time cube. More...
ArrayList	getInstantSpeed ()	! Return the ArrayList<float> attribut that represents the instant speed between two consecutive points in the extracted spatio-temporal data from the GPX file. More...
float	getMinimumInstantSpeed ()	! Return the float attribut that represents the minimum instant speed in the extracted spatio-temporal data from the GPX file. More...

Figure 24 : Capture d'écran de la documentation d'une des classes du projet (générée avec Doxygen)

Les balises et tags utilisés dans le code source furent également compatibles avec les balises de documentation automatique de Visual Studio 2017 permettant ainsi de visualiser la documentation d'une fonction ou d'une classe sans nécessairement aller jusqu'au fichier source et en laissant simplement la souris sur l'appel de la fonction ou sur un objet de la classe en question. Cela fut particulièrement utile lors du développement logiciel de l'application.

IV. Vidéos de démonstration

Un ensemble de vidéos de démonstration fut également réalisé afin de montrer les différentes techniques de visualisation et d'exploration mises en place pour les deux scénarios décrits précédemment : le scénario de l'entraînement sportif du cycliste et celui de néOCampus. Ces vidéos sont disponibles à l'adresse suivante :

https://drive.google.com/open?id=1vCtChRveRx_mUPYsK4UNP-9kZZ5TCSMM

Ces vidéos furent prises la dernière semaine du stage de manière à pouvoir visualiser la dernière version de l'application. De cette manière, des améliorations furent apportées à l'application à partir des retours utilisateurs et des personnes ayant visionné les vidéos précédentes.

Conclusion

Durant ce stage, nous avons exploré différentes solutions existantes pour la visualisation et l'exploration de DST à travers une bibliographie détaillée. Nous avons étudié et pris en compte les limitations et avantages de ces solutions pour proposer un système original basé sur l'utilisation simultanée des interactions tangibles, de la robotique et des interactions tactiles pour accéder, interpréter et manipuler des ensembles de DST. L'utilisation d'un robot permet de mettre en évidence physiquement et spatialement un sous-ensemble des DST tout en autorisant l'utilisateur à le manipuler (saisir, tenir, déplacer) pour accéder à un plus grand nombre d'informations. Contrairement aux outils classiques d'interaction, cette interaction physique et directe avec le robot ouvre un large champ d'actions possibles sur le système. Par l'utilisation d'un écran de visualisation des données, support physique pour le robot, il nous est ainsi possible de générer la trajectoire du robot tout en visualisant les DST ciblées par l'utilisateur.

L'état actuel du système ne permet aujourd'hui d'analyser qu'un unique ensemble de données. Ainsi, nous prévoyons de rendre le système capable d'accueillir plusieurs ensembles de données simultanément pour comparer les données relatives à deux entités suivies sur des parcours partageant une même zone géographique tels que deux cyclistes s'entraînant le même jour ou deux occurrences du même parcours par un même cycliste. Nous envisageons donc d'intégrer la gestion d'une flotte de robots, chacun représentant un ensemble de données distinct. De plus, il serait intéressant d'explorer des technologies émergentes telles que la réalité augmentée ou la réalité mixte. Celles-ci permettraient de mettre en évidence certaines données en conservant le contexte cartographique décrit précédemment : cet enrichissement permettrait de mettre en évidence les différents points d'intérêt des DST, ou d'autoriser la visualisation d'informations supplémentaires sans surcharger l'utilisateur par rapport au nombre d'informations visibles au même instant ou au nombre d'opérations nécessaires pour accéder à une information souhaitée.

À l'issue de ce stage deux publications furent soumises. La première fut soumise et acceptée au début du stage à la conférence « MultimodalVis 2018 ». Cette publication est disponible à l'adresse suivante : https://multimodalvis.github.io/papers/AVI_2018_paper_146.pdf. Un second article fut soumis à la conférence « IHM 2018 ». Il fut accompagné d'un poster et d'une démonstration vidéo présentant l'application décrite dans cet article. L'envoi des évaluations aux auteurs aura lieu le 10 septembre 2018 et une version finale sera alors à renvoyer.

En supplément de la réalisation d'un prototype fonctionnel, nous avons participé activement au projet néOCampus en assistant aux différentes réunions, salons (notamment le salon Enova 2018) et présentations de ce projet pluridisciplinaire (incluant des présentations des projets de l'équipe Elipse et de notre projet sur l'exploration de DST compatible avec le projet néOCampus de plus grande envergure).

D'un point de vue personnel, j'ai beaucoup appris lors de ce stage. J'ai pu apprendre comment utiliser Unity et comment tirer parti de ces avantages comme de ses inconvénients. En effet, Unity n'est pas adapté à la création et destruction dynamique d'objets, et est généralement utilisé pour la création de jeux vidéo qui suivent un scénario précis avec des paramètres prédéterminés. Dans notre cas, beaucoup de variables furent imprévisibles comme par exemple la forme ou la quantité des DST à extraire du fichier GPX ou encore la position du robot sur l'écran d'interaction. Il fut donc nécessaire

de mettre en place des algorithmes prenant en compte ces paramètres imprévisibles pour s'assurer qu'en cas d'absence d'informations sur ces variables le programme puisse continuer à fonctionner.

De plus, en créant un programme modulaire et générique, il fut possible de prendre en compte toutes les modifications régulières demandées par mes encadrants, bien que je fus parfois frustrée de ne pas pouvoir tester des solutions mises en place avant qu'elles ne soient abandonnées. En structurant le code source de manière à subdiviser le rôle de chaque fonctionnalité de l'application, il fut alors possible de regrouper des fonctions en classes dont chacune eut un rôle essentiel dans l'application. Par exemple, une classe fut créée pour l'extraction des données, une autre pour la communication avec le serveur de localisation, une autre pour l'interface graphique, etc.

Mes encadrants n'ayant aucune expertise en robotique, ils me donnèrent une grande liberté sur la proposition et l'évaluation de solutions pour localiser le robot Ozobot sur l'écran d'interaction et le contrôler. Ce fut une partie de mon stage qui fut très enrichissante. Cela me permit de développer ma réflexion en ingénierie pour améliorer des concepts existants et les adapter à notre cas d'utilisation (comme par exemple mettre en place un cercle de contrôle autour du robot pour ne pas perturber sa fonction de suivi de ligne, ou encore améliorer un système de localisation en évaluant ces limitations existantes et en mettant en place des solutions pour y remédier).

Enfin, j'ai pu apprécier tout au long de mon stage le soutien et l'aide apportés par des personnes extérieures au projet comme par exemple l'aide des stagiaires et de Florent Cabric, ingénieur dans l'équipe Elipse, avec un retour régulier sur des tests d'utilisabilité de l'application m'offrant la possibilité de proposer de nouvelles techniques d'interaction ou de visualisation des données, ou tout simplement d'améliorer le prototype proposé lors de ce stage. Des stagiaires spécialisés dans le design et le graphisme m'apprirent également beaucoup sur la manière de présenter mon travail par des schémas ou des dessins. Elles me montrèrent comment utiliser différents logiciels de la suite Adobe pour obtenir le meilleur résultat possible. Ce logiciel fut très utilisé à la fin de mon stage, notamment pour la publication à la conférence « IHM 2018 ».

Références

1. Bach, B., Dragicevic, P., Archambault, D., Hurter, C., & Carpendale, S. (2014). A Review of Temporal Data Visualizations Based on Space-Time Cube Operations. Présenté à Eurographics Conference on Visualization. Consulté à l'adresse <https://hal.inria.fr/hal-01006140/document>
2. Geographic coordinate system. (2018). In *Wikipedia*. Consulté à l'adresse https://en.wikipedia.org/w/index.php?title=Geographic_coordinate_system&oldid=855259219
3. Gerndt, R., & Krupop, S. (2010). EcoBe! Mixed Reality Robot Kit-An Entry-Level System for Teaching Cooperative Robotics.
4. Hägerstraand, T. (1970). What About People in Regional Science? *Papers in Regional Science*, 24(1), 7-24. <https://doi.org/10.1111/j.1435-5597.1970.tb01464.x>
5. Le Goc, M., Kim, L. H., Parsaei, A., Fekete, J.-D., Dragicevic, P., & Follmer, S. (2016). Zoids: Building Blocks for Swarm User Interfaces. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology* (p. 97–109). New York, NY, USA: ACM. <https://doi.org/10.1145/2984511.2984547>
6. Li, X., Çöltekin, A., & Kraak, M.-J. (2010). Visual Exploration of Eye Movement Data Using the Space-time-cube. In *Proceedings of the 6th International Conference on Geographic Information Science* (p. 295–309). Berlin, Heidelberg: Springer-Verlag. Consulté à l'adresse <http://dl.acm.org/citation.cfm?id=1887961.1887982>
7. Özgür, A., Johal, W., Mondada, F., & Dillenbourg, P. (2017). Haptic-Enabled Handheld Mobile Robots: Design and Analysis. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (p. 2449–2461). New York, NY, USA: ACM. <https://doi.org/10.1145/3025453.3025994>
8. Özgür, A., Lemaignan, S., Johal, W., Beltran, M., Briod, M., Pereyre, L., ... Dillenbourg, P. (2017). Cellulo: Versatile Handheld Robots for Education. In *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction* (p. 119–127). New York, NY, USA: ACM. <https://doi.org/10.1145/2909824.3020247>
9. Ozobot | Robots to code, create, and connect with. (s. d.). Consulté 23 août 2018, à l'adresse <https://ozobot.com/>
10. Tominski, C., Schumann, H., Andrienko, G., & Andrienko, N. (2012). Stacking-Based Visualization of Trajectory Attribute Data. *IEEE Transactions on Visualization and Computer Graphics*, 18(12), 2565-2574. <https://doi.org/10.1109/TVCG.2012.265>
11. Yusof, N., Zurita-Milla, R., Kraak, M.-J., & Retsios, B. (2016). Interactive discovery of sequential patterns in time series of wind data. *International Journal of Geographical Information Science*, 30(8), 1486-1506. <https://doi.org/10.1080/13658816.2015.1135928>